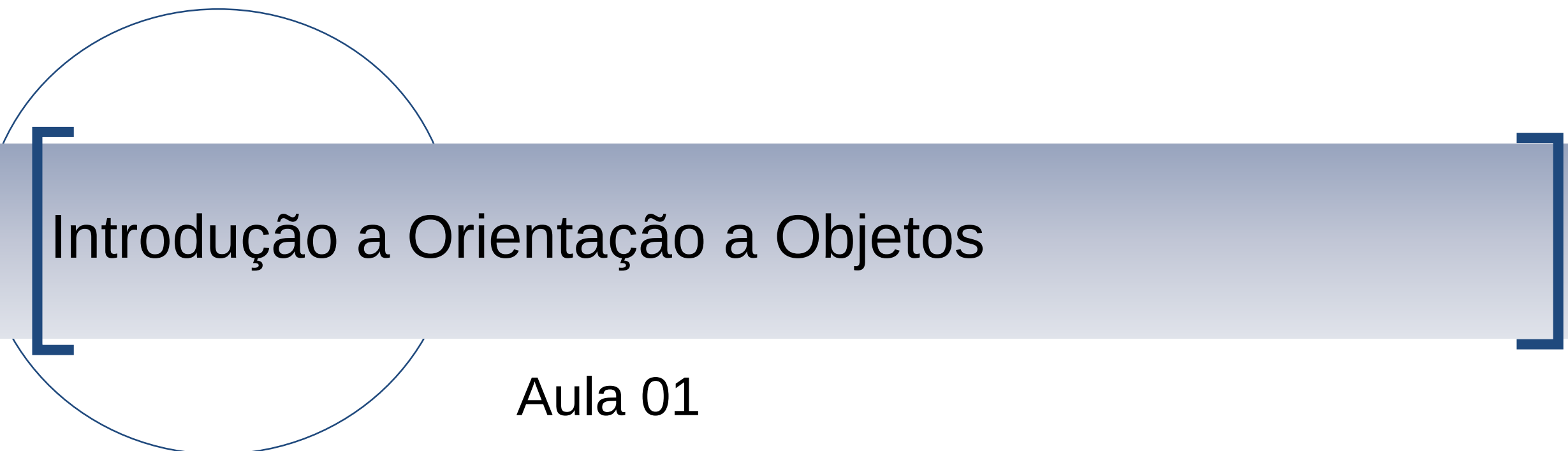
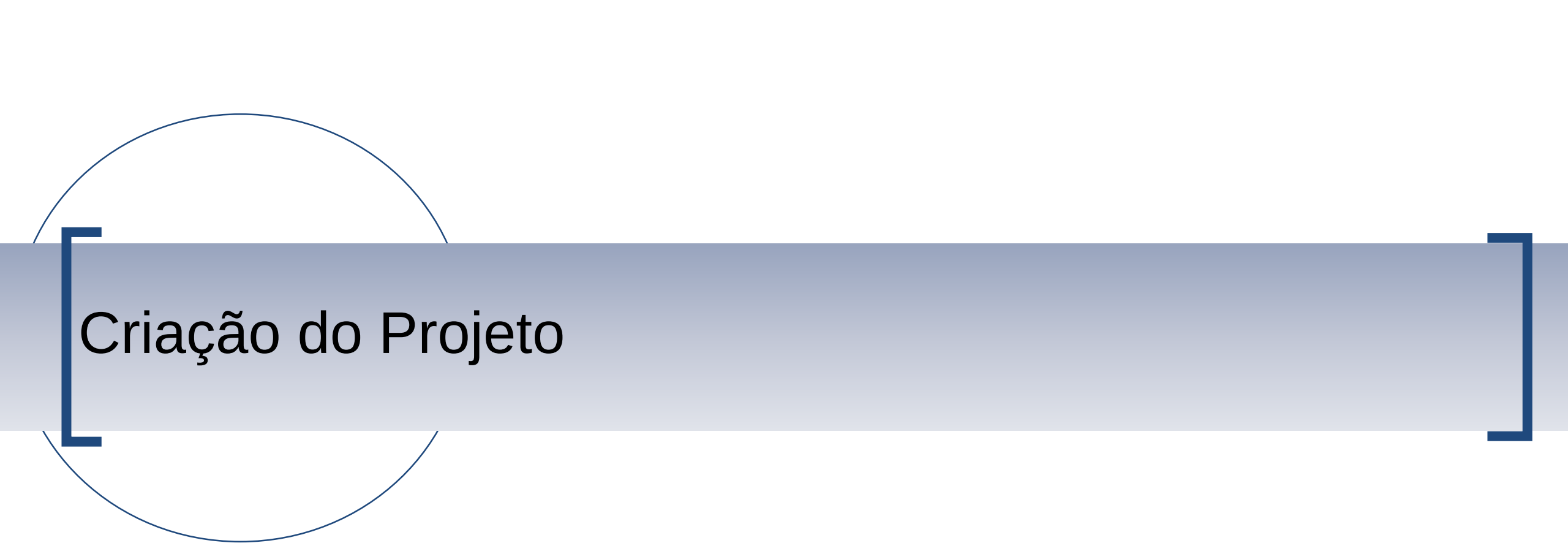




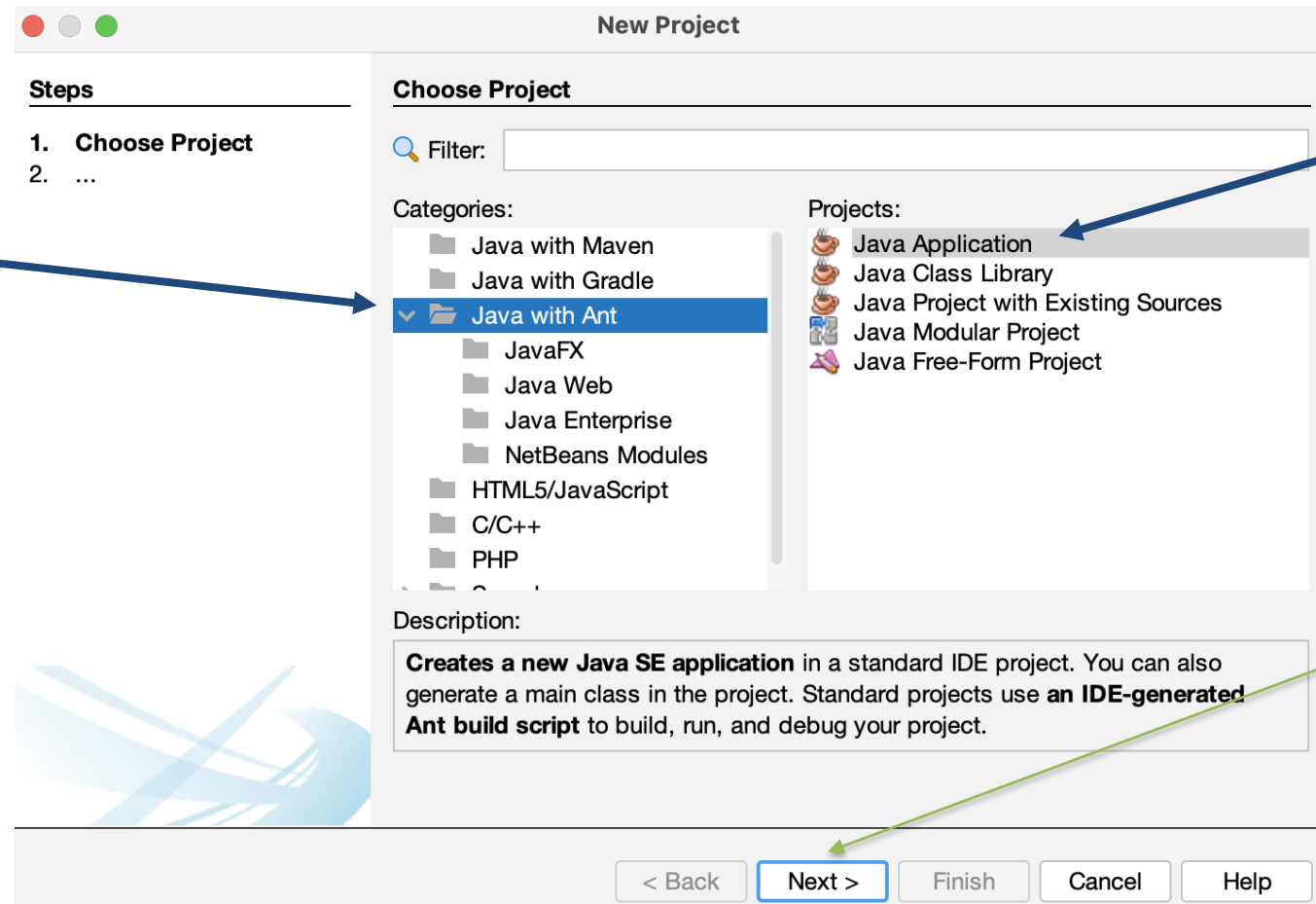
Introdução a Orientação a Objetos

Aula 01



Criação do Projeto

Criação do Projeto



Selecione o
Java with Ant

Selecione o
Java Application

clique

Criação do Projeto

New Java Application

Steps

1. Choose Project
2. **Name and Location**

Name and Location

Project Name:

Project Location:

Project Folder:

☐ Use Dedicated Folder for Storing Libraries

Libraries Folder:

Different users and projects can share the same compilation libraries (see Help for details).

☐ Create Main Class

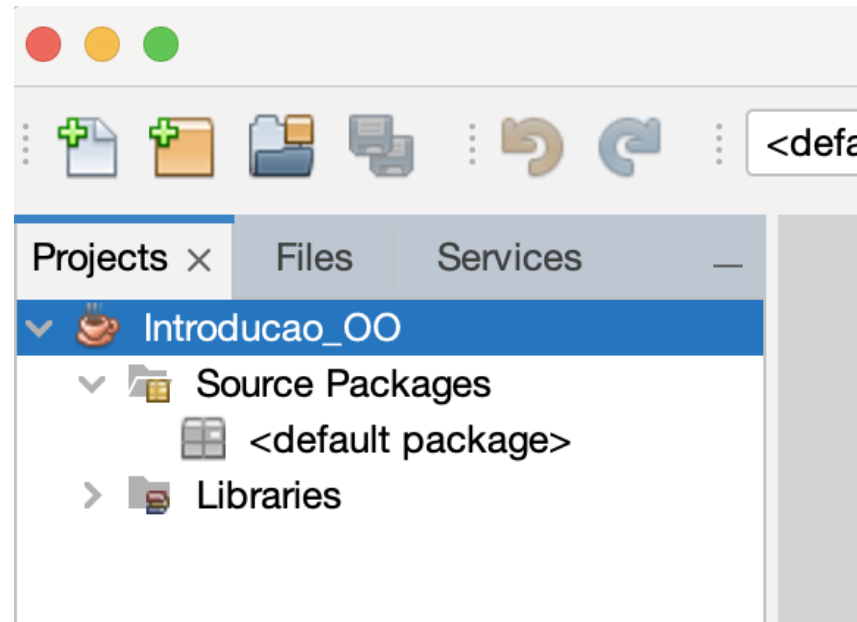
Digite o nome
Exemplo1_OO

Desmarque esta
opção

clique

Criação do Projeto

Projeto Criado
com Sucesso !!!

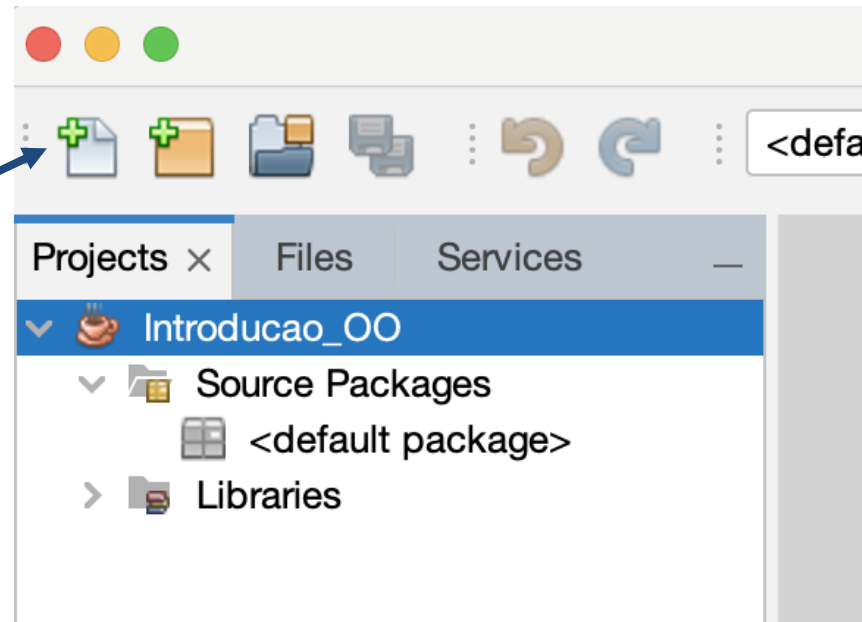


A decorative graphic consisting of a thin blue circle on the left side, partially obscured by a thick, light blue horizontal bar that spans the width of the slide. The bar has dark blue L-shaped brackets at its left and right ends.

Adicionando um JFrame ao Projeto

Adicionando um JFrame ao Projeto

Clique para
adicionar um item
ao projeto



Adicionando um JFrame ao Projeto

Steps

1. **Choose File Type**
2. ...

Choose File Type

Project: Introducao_OO

Filter:

Categories:

- Java
- Swing GUI Forms**
- JavaBeans Objects
- AWT GUI Forms
- Unit Tests
- Web
- JavaServer Faces
- Struts
- Spring Framework

File Types:

- JDialog Form
- JFrame Form**
- JInternalFrame Form
- JPanel Form
- JApplet Form
- Bean Form
- Application Sample Form
- MDI Application Sample Form
- OK / Cancel Dialog Sample Form

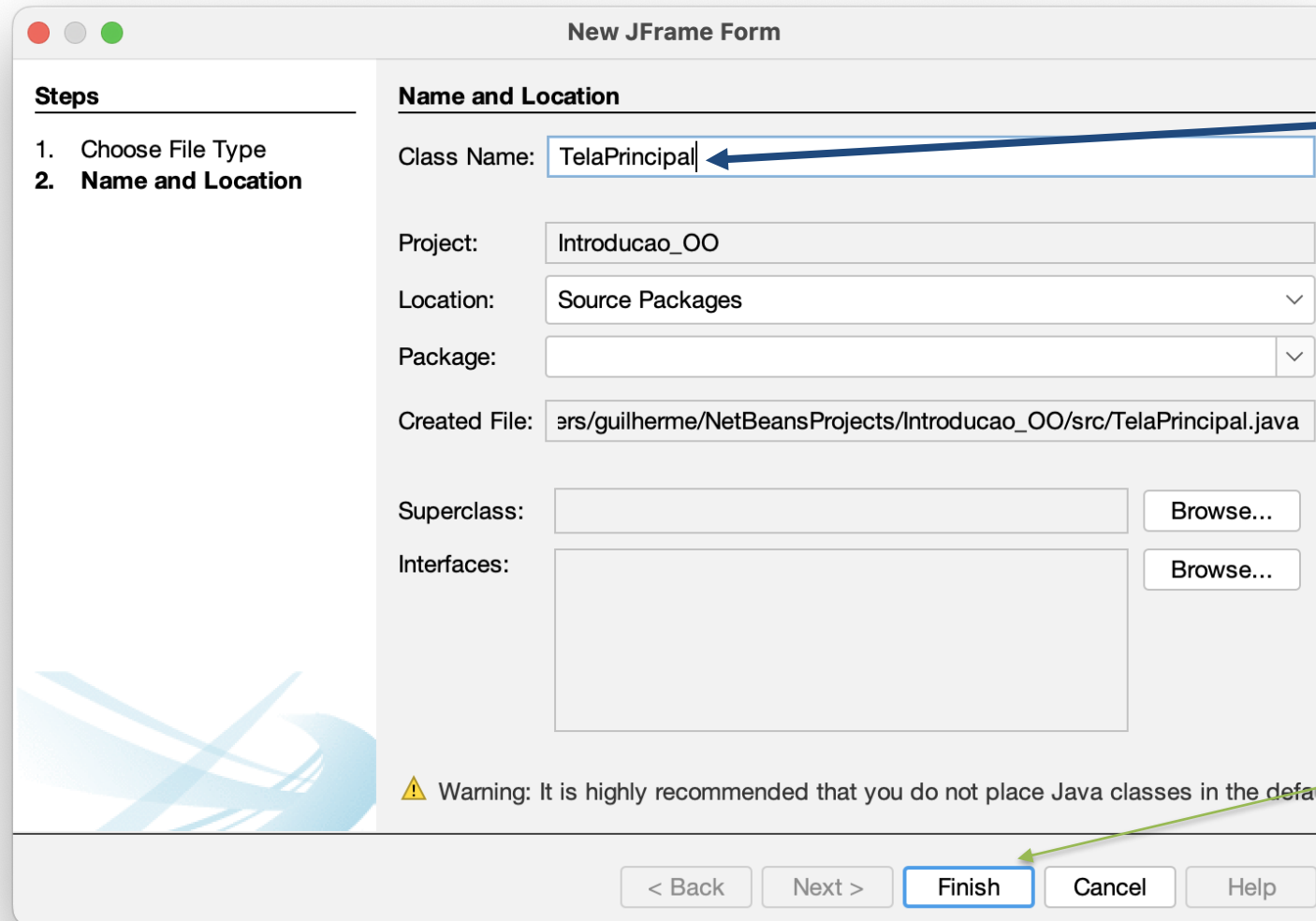
Description:

Creates a new JFC (Swing) Frame. Frames are typically used as standalone top-level windows as the main user interface to the application. Most Swing applications are built starting from this form.

Annotations:

- Selezione Swing GUI Forms
- Selezione JFrame Form
- clique

Adicionando um JFrame ao Projeto



Steps

1. Choose File Type
2. **Name and Location**

Name and Location

Class Name:

Project:

Location:

Package:

Created File:

Superclass:

Interfaces:

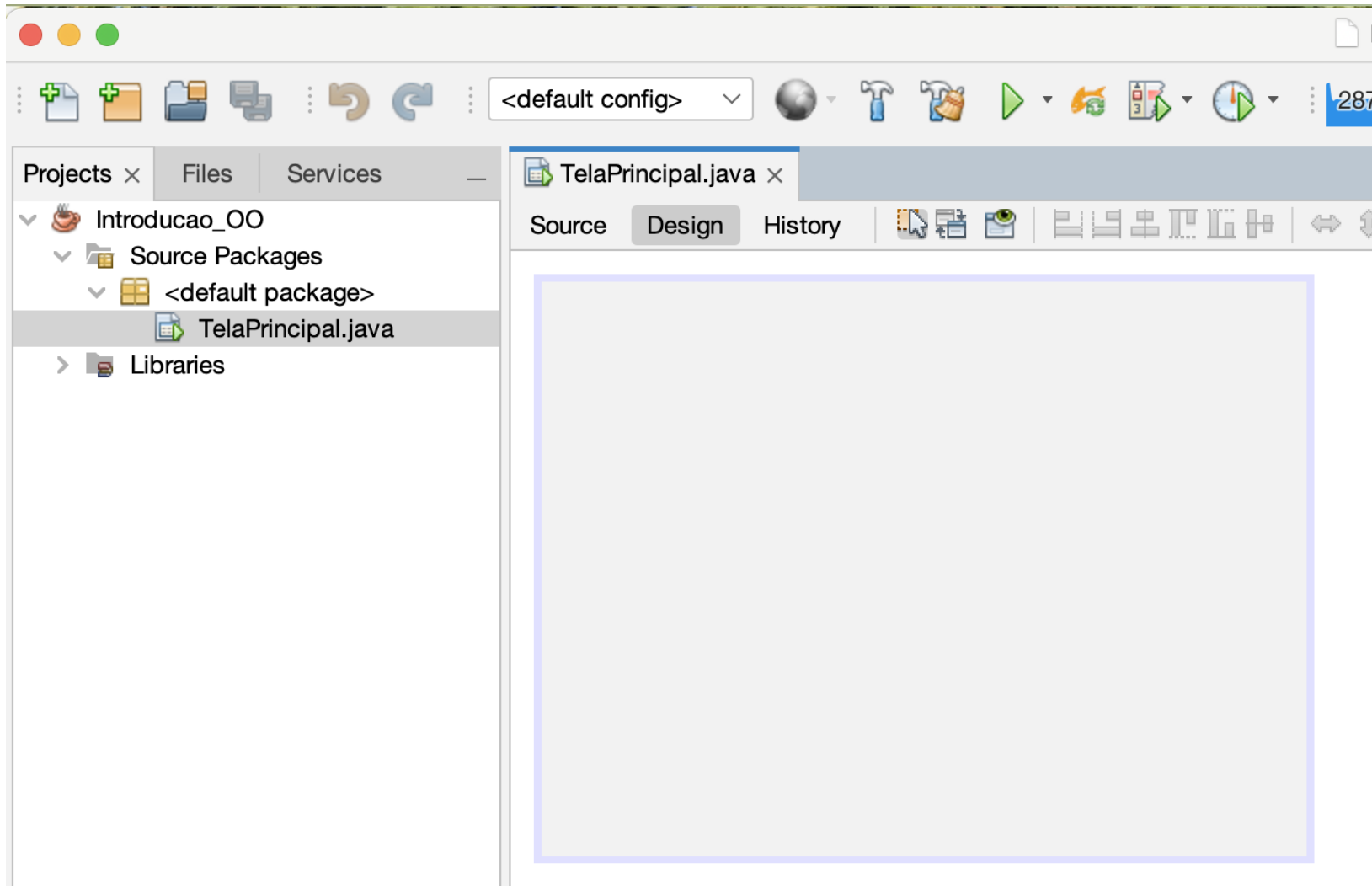
Warning: It is highly recommended that you do not place Java classes in the default package.

< Back Next > **Finish** Cancel Help

Digite o nome
TelaPrincipal

clique

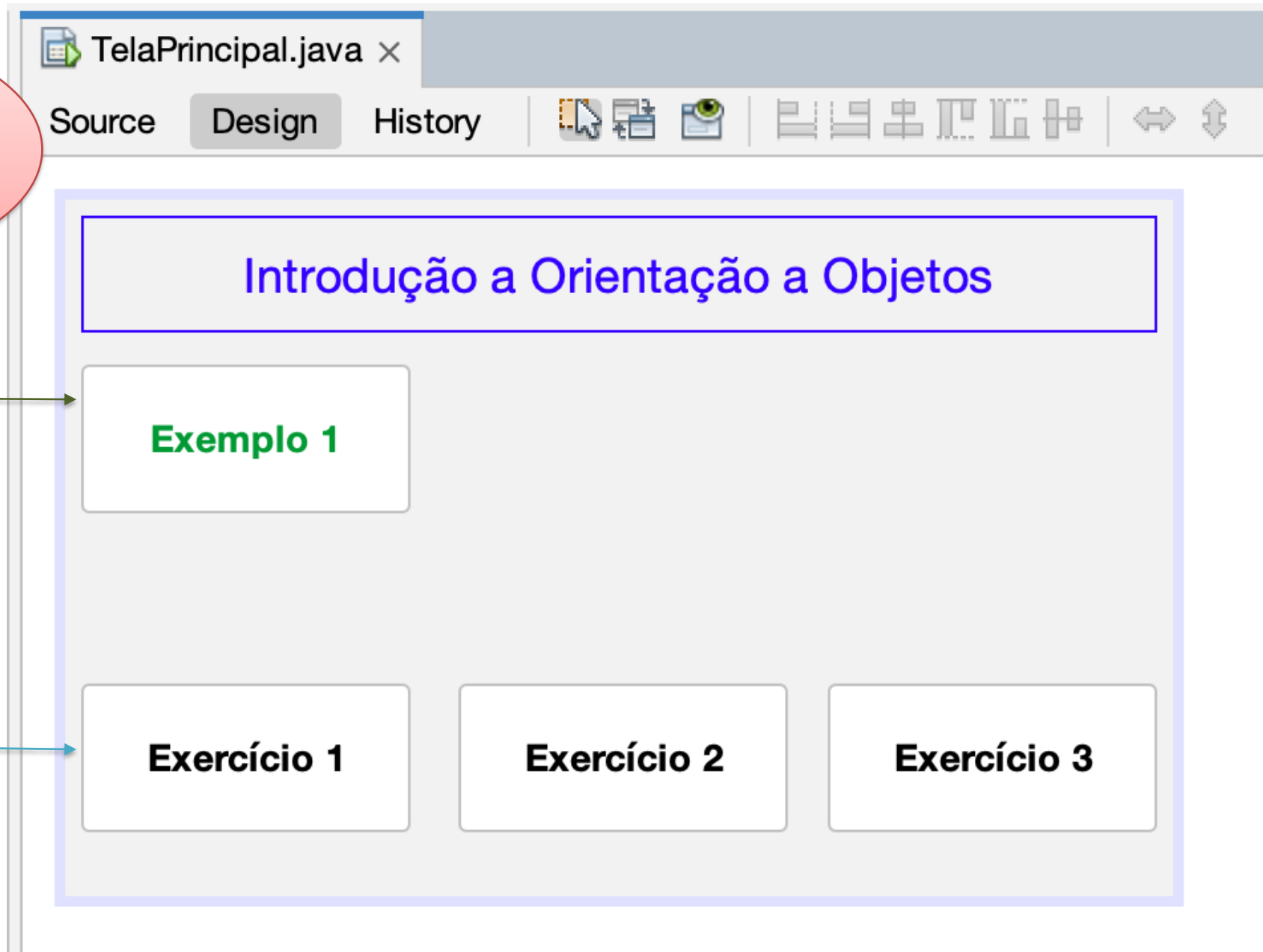
Adicionando um JFrame ao Projeto

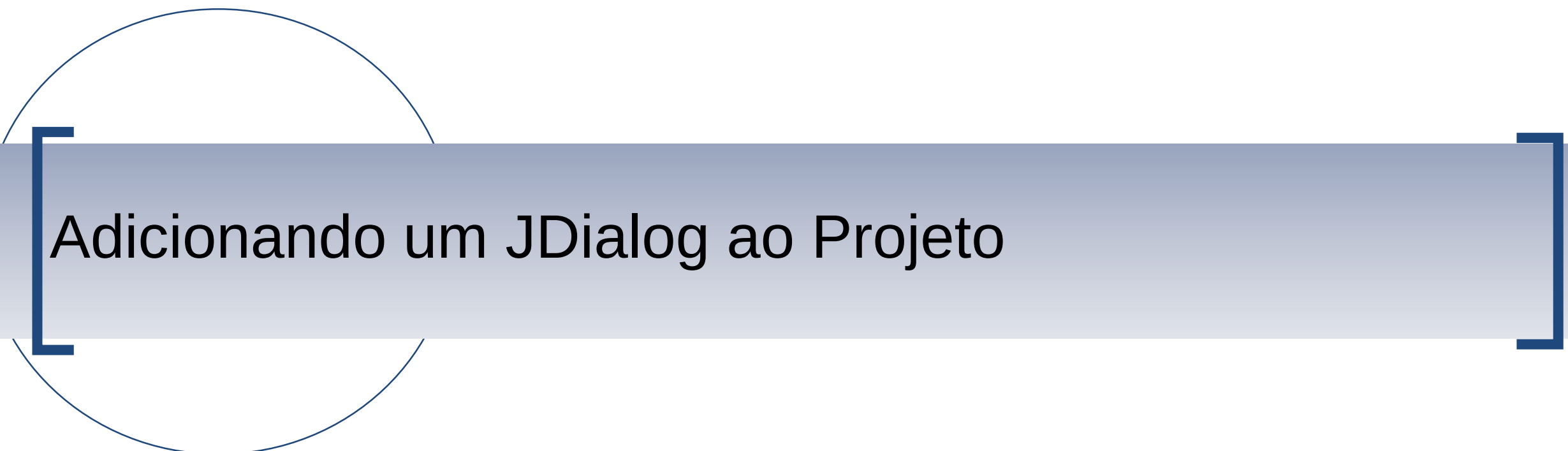


Adicione
1 label, e 4 botões

btnExemplo1

btnExercicio1
btnExercicio2
btnExercicio3

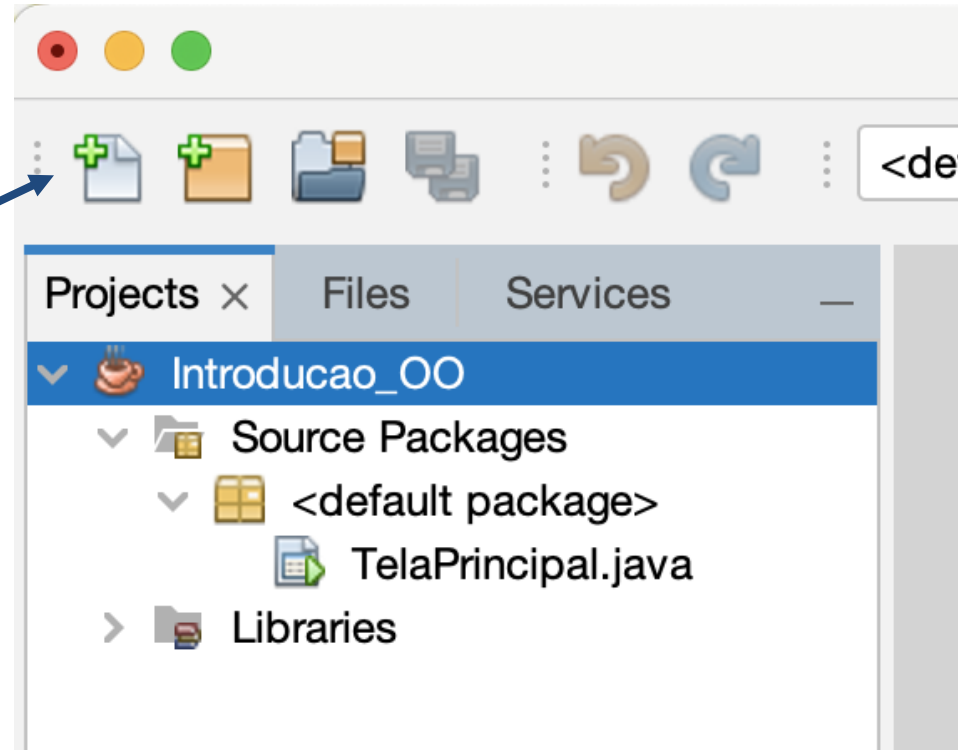


A decorative graphic consisting of a thin blue circle on the left side and a thick blue horizontal bar extending across the middle of the slide. The title text is centered within the blue bar.

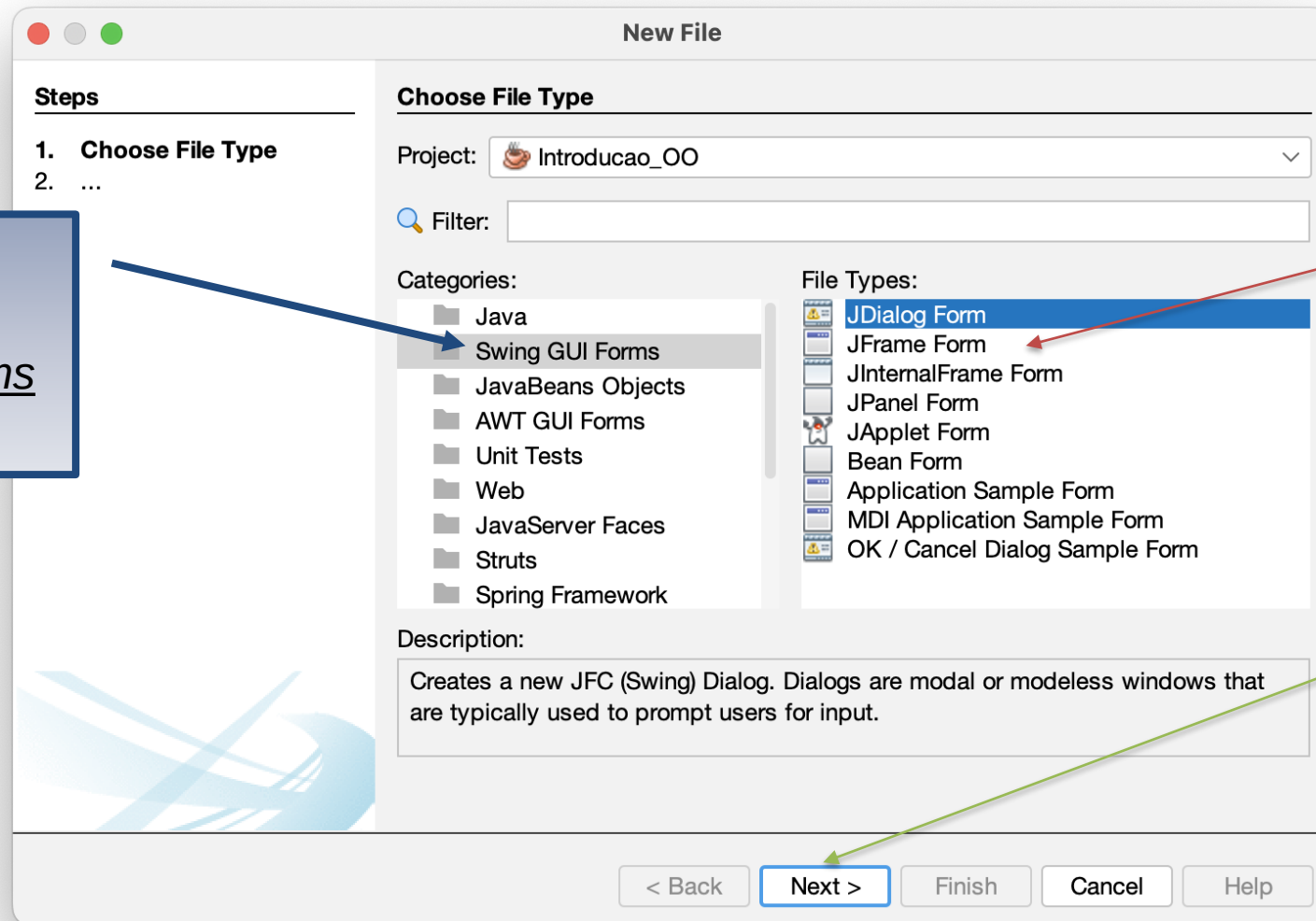
Adicionando um JDialog ao Projeto

Adicionando um JDialog ao Projeto

Clique para
adicionar um item
ao projeto



Adicionando um JDialog ao Projeto

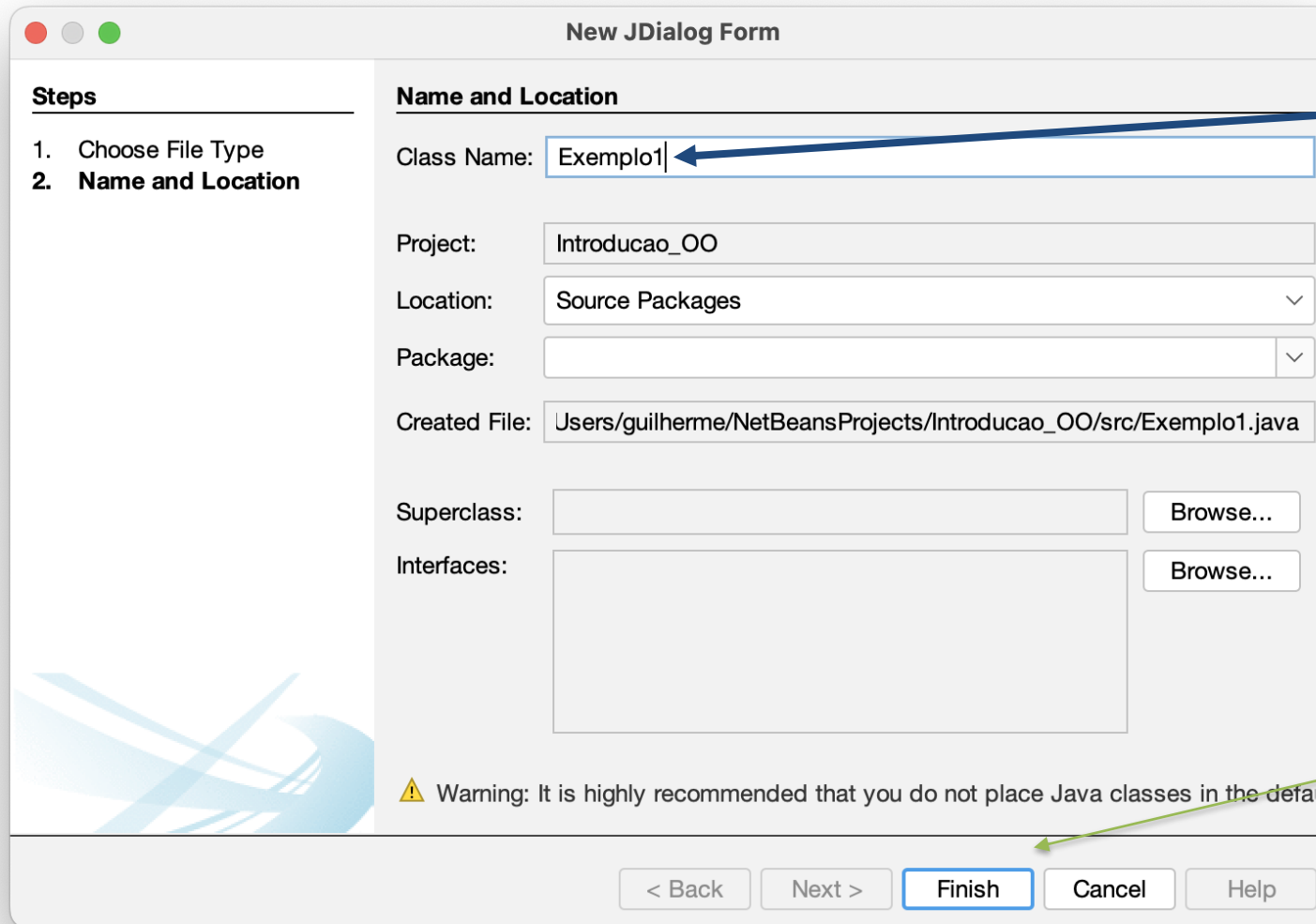


Selecione
Swing GUI Forms

Selecione
JDialog Form

clique

Adicionando um JDialog ao Projeto



Steps

1. Choose File Type
2. **Name and Location**

Name and Location

Class Name:

Project:

Location:

Package:

Created File:

Superclass:

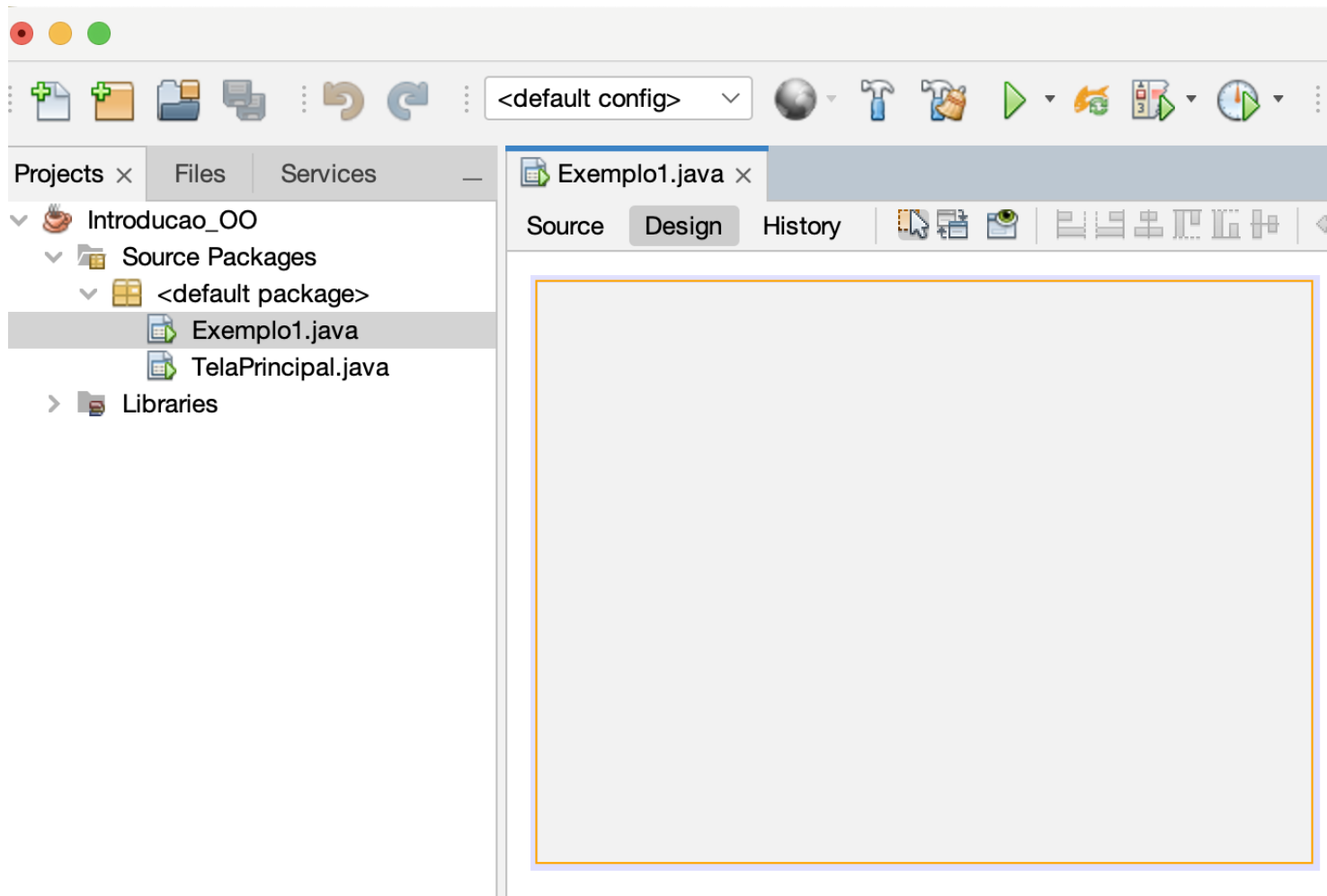
Interfaces:

⚠ Warning: It is highly recommended that you do not place Java classes in the default package.

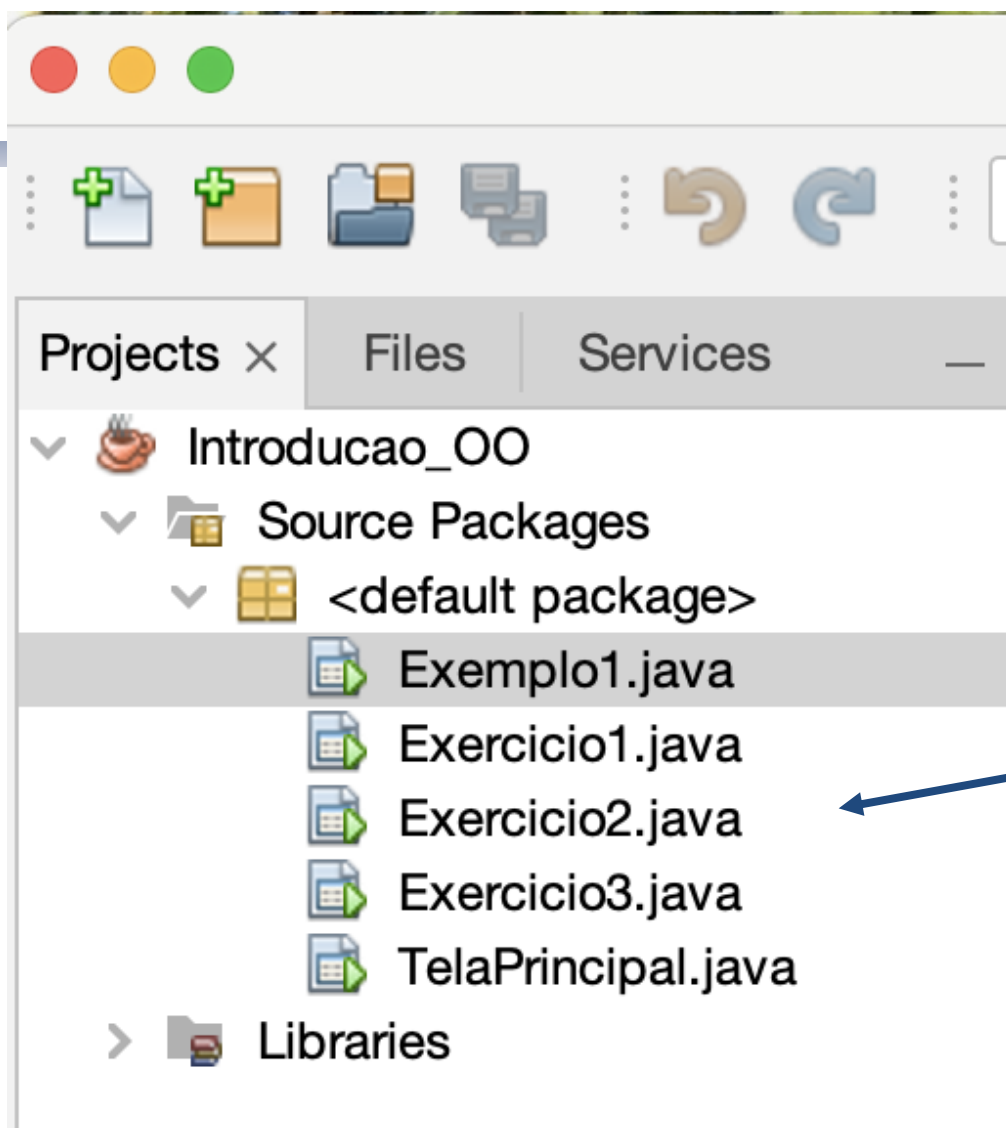
Digite o nome
Exemplo1

clique

Adicionando um JDialog ao Projeto

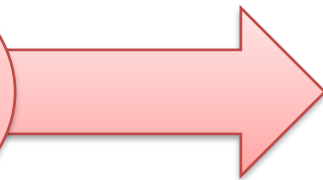


Formulário
adicionado



Repita o Procedimento, e
adicione 3 JDialogs para
exercícios

Construa este formulário



Exemplo1.java x

Source Design History

Exemplo 1 - Calculo da Média de um Aluno

Digite o nome do aluno

Digite a nota 1

Digite a nota 2

Calcular utilizando Programação Estruturada

Calcular utilizando a Programação Orientada a Objetos

Média **resultado**

Situação **resultado**

Other Components - Navigator ×

Form Exemplo1

Other Components

[-] [JDialog]

- label jLabel1 [JLabel]
- label jLabel2 [JLabel]
- label jLabel3 [JLabel]
- label jLabel4 [JLabel]
- label jLabel5 [JLabel]
- label jLabel6 [JLabel]
- txtNome [JTextField]
- txtNota1 [JTextField]
- txtNota2 [JTextField]
- [OK] btnCalcularPE2 [JButton]
- [OK] btnCalcularOO [JButton]
- label labelMedia [JLabel]
- label labelSituacao [JLabel]

Exemplo1.java ×

Source Design History

Exemplo 1 - Calculo da Média de um Aluno

Digite o nome do aluno

Digite a nota 1

Digite a nota 2

Calcular utilizando Programação Estruturada

Calcular utilizando a Programação Orientada a Objetos

Média **resultado**

Situação **resultado**

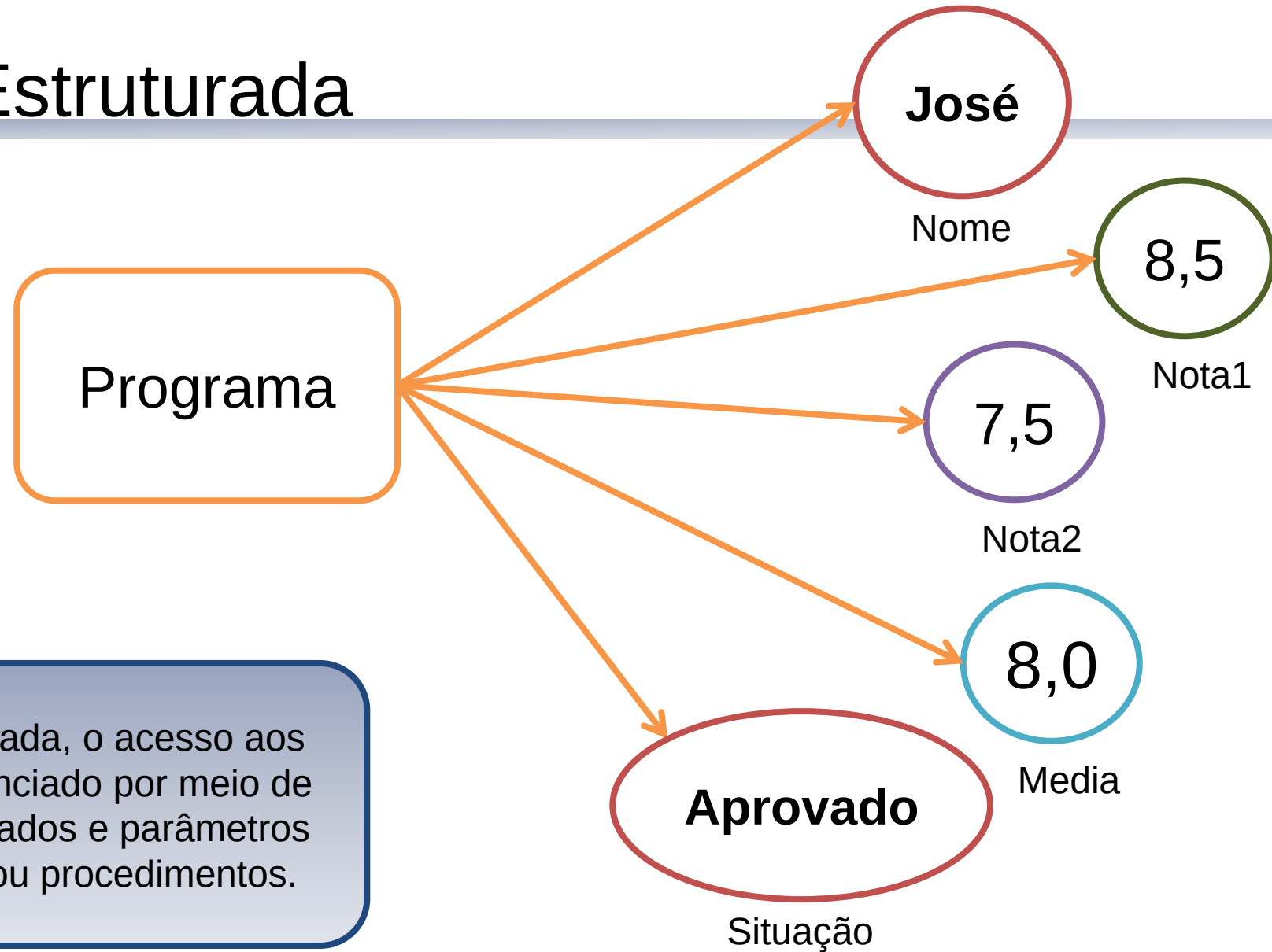
Programação Estruturada

- O que é a Programação Estruturada ?
 - É um paradigma de programação que se baseia na organização lógica e sequencial do código, com ênfase no uso de estruturas de controle de fluxo bem definidas.
 - Buscando dividir o programa em blocos de código independentes e bem estruturados, facilitando o desenvolvimento, a manutenção e a compreensão do software.

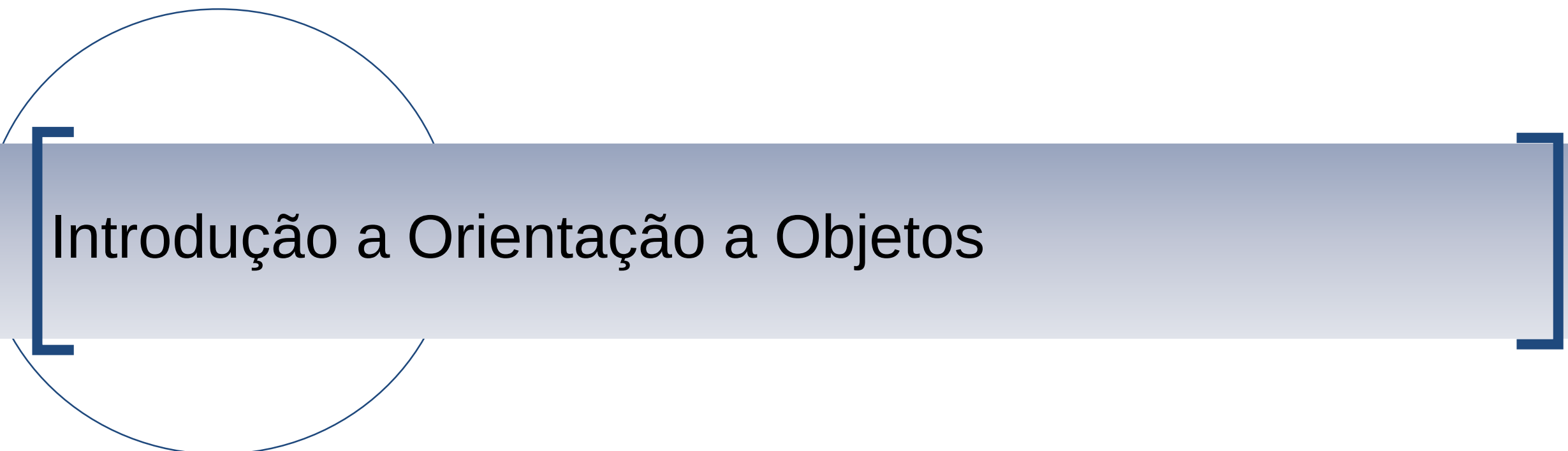
```
private void btnCalcularPE2ActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
  
    String nome, situacao;  
    double nota1, nota2, media;  
  
    nome = txtNome.getText();  
    nota1 = Double.parseDouble( s:txtNota1.getText());  
    nota2 = Double.parseDouble( s:txtNota2.getText());  
  
    media = (nota1 + nota2) / 2;  
    labelMedia.setText("" + media);  
  
    if (media >= 6)  
    {  
        labelSituacao.setText( text: "Aprovado");  
    }  
    else  
    {  
        labelSituacao.setText( text: "Reprovado");  
    }  
}
```

É um paradigma de programação que se baseia na organização lógica e sequencial do código

Programação Estruturada

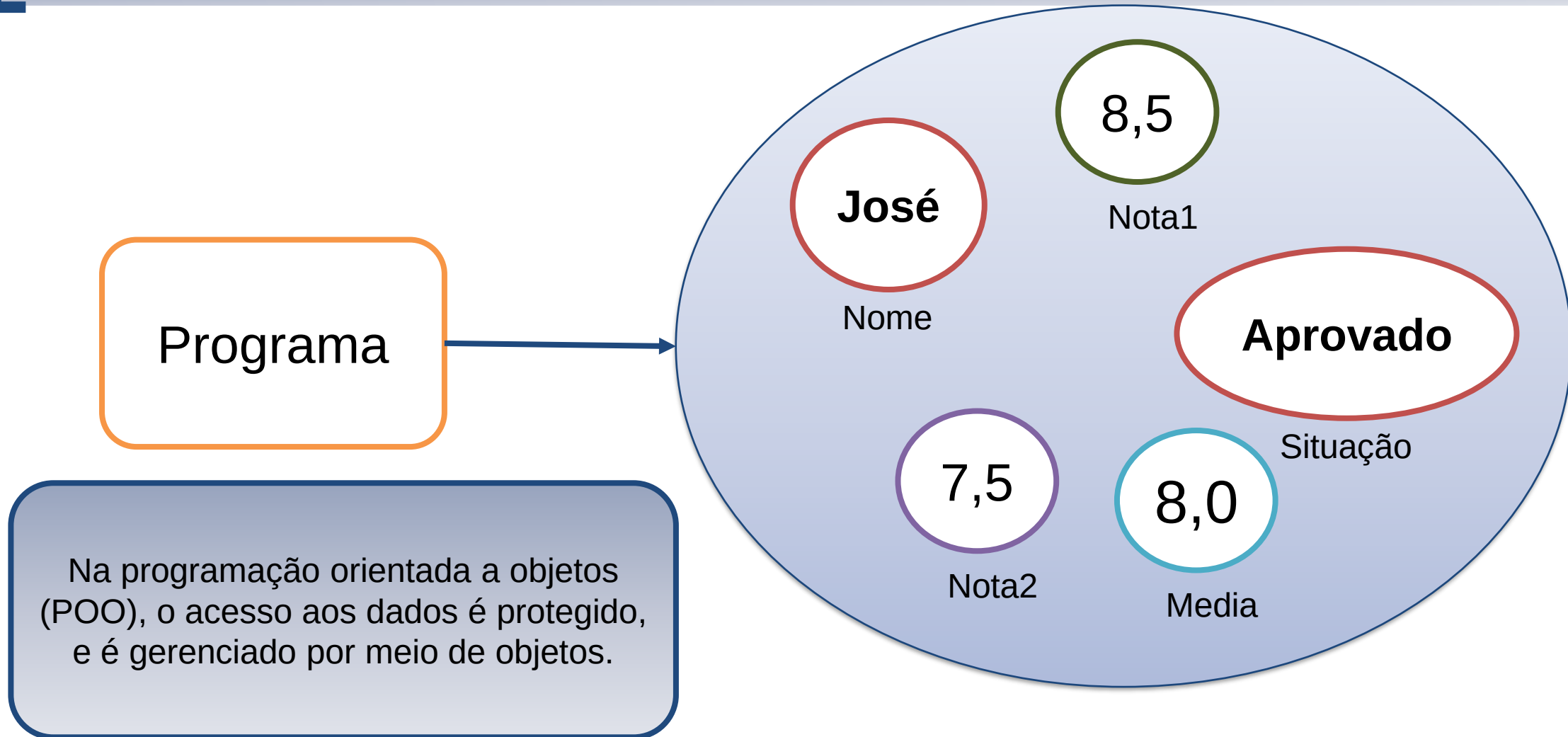


Na programação estruturada, o acesso aos dados é geralmente gerenciado por meio de variáveis, estruturas de dados e parâmetros passados para funções ou procedimentos.

A decorative graphic consisting of a thin blue circle on the left and a solid blue horizontal bar extending across the middle of the slide. The title text is centered within the blue bar.

Introdução a Orientação a Objetos

Introdução a Orientação a Objetos



Introdução a Orientação a Objetos

■ Classes

- O programa referente ao Exemplo 1 representa um “**aluno**” que possui as seguintes características:
 - Nome
 - Nota1
 - Nota2
 - Media
 - Situação

Exemplo1.java x

Source Design History

Exemplo 1 - Calculo da Média de um Aluno

Digite o nome do aluno

Digite a nota 1

Digite a nota 2

Calcular utilizando Programação Estruturada

Calcular utilizando a Programação Orientada a Objetos

Média **resultado**

Situação **resultado**

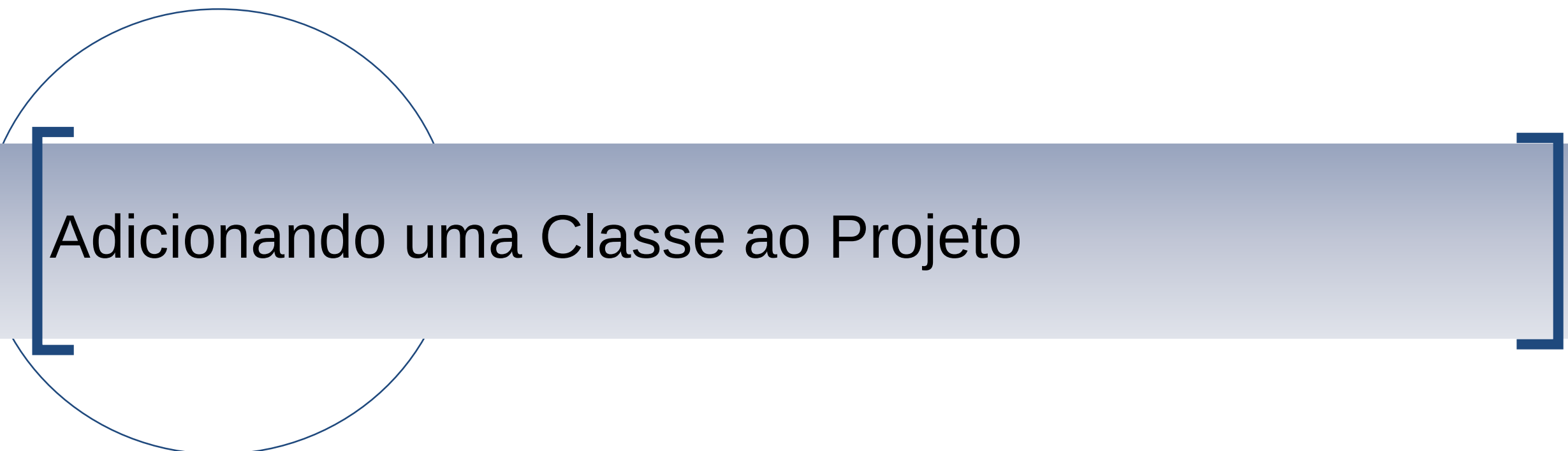
Introdução a Orientação a Objetos

■ Classes

- Através das classes vamos organiza o código em objetos que representam entidades do mundo real, contendo as suas características ou dados (atributos) e comportamentos (métodos).
- Portanto uma classe *é uma estrutura que define os atributos (dados) e métodos (comportamentos) que os objetos da classe terão.*
 - É uma espécie de modelo ou plano para a criação de objetos.

Introdução a Orientação a Objetos

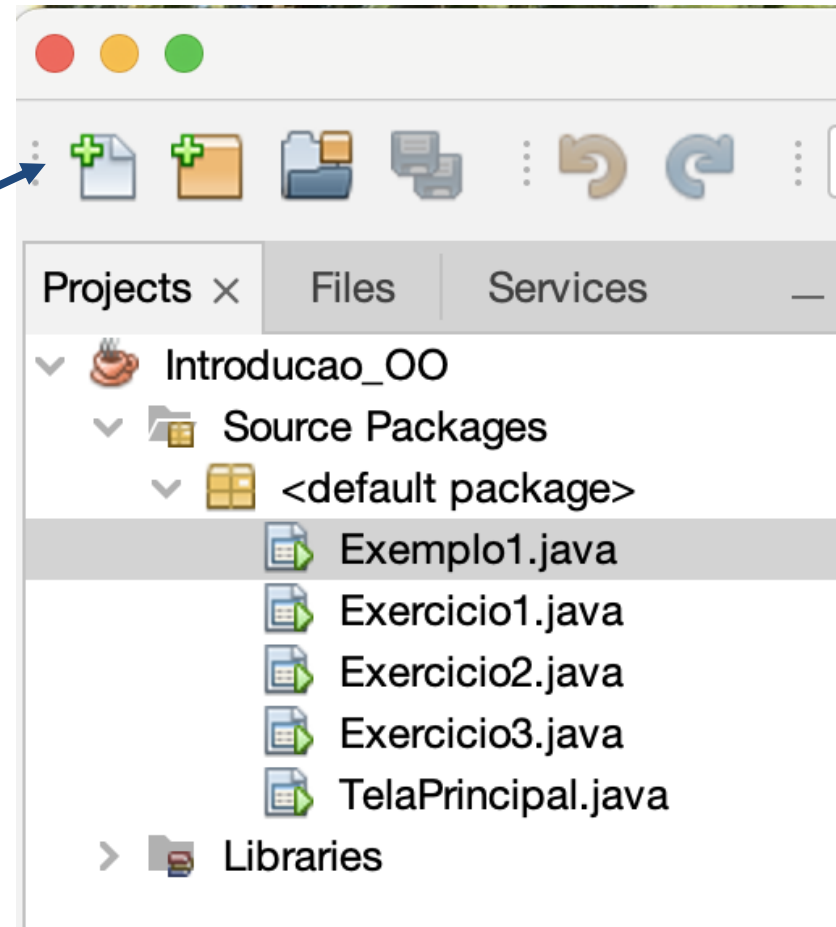
- Vamos adicionar um classe ao nosso Projeto
 - Com o objetivo de representar os Alunos e suas respectivas características.

A decorative graphic consisting of a thin blue circle on the left side, partially enclosed by a thick blue bracket that extends horizontally across the top and bottom of the slide's main content area.

Adicionando uma Classe ao Projeto

Adicionando uma Classe ao Projeto

Clique para adicionar
uma nova classe



Adicionando uma Classe ao Projeto

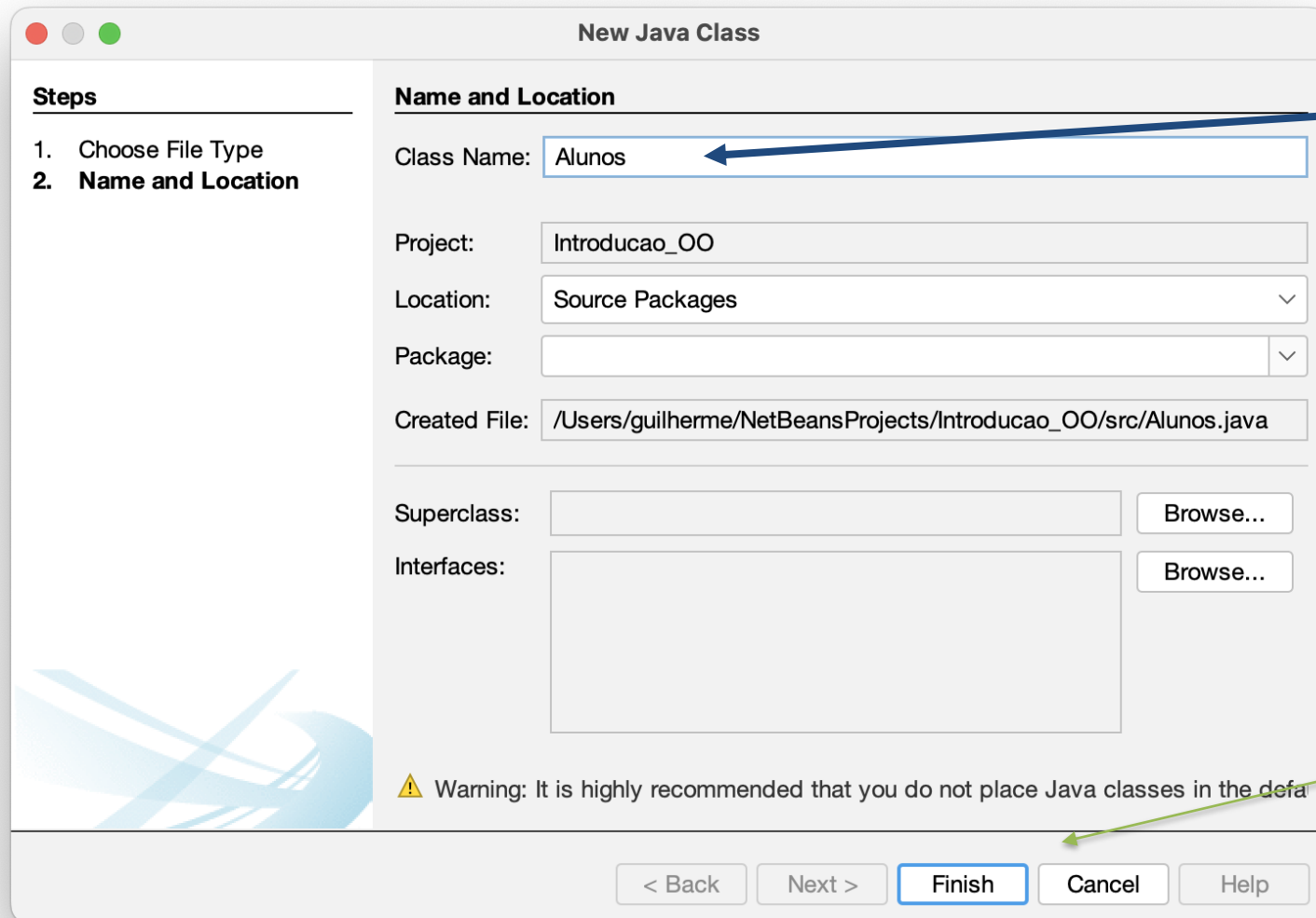
The screenshot shows the 'New File' dialog with the following elements:

- Steps:** 1. Choose File Type, 2. ...
- Choose File Type:**
 - Project: Introducao_OO
 - Filter: (empty)
 - Categories: Java (selected), Swing GUI Forms, JavaBeans Objects, AWT GUI Forms, Unit Tests, Web, JavaServer Faces, Struts, Spring Framework.
 - File Types: Java Class (selected), Java Interface, Java Enum, Java Annotation Type, Java Exception, Java Package Info, Java Module Info, JApplet, Applet, Java Main Class, Java Singleton Class.
 - Description: Creates a new plain Java class. This template is useful for creating new non-visual classes.
- Buttons:** < Back, Next > (highlighted), Finish, Cancel, Help.

Annotations:

- Blue box: Selecione Java (points to the 'Java' category in the Categories list).
- Red box: Selecione Java Class (points to 'Java Class' in the File Types list).
- Green box: clique (points to the 'Next >' button).

Adicionando uma Classe ao Projeto



New Java Class

Steps

1. Choose File Type
2. **Name and Location**

Name and Location

Class Name:

Project:

Location:

Package:

Created File:

Superclass:

Interfaces:

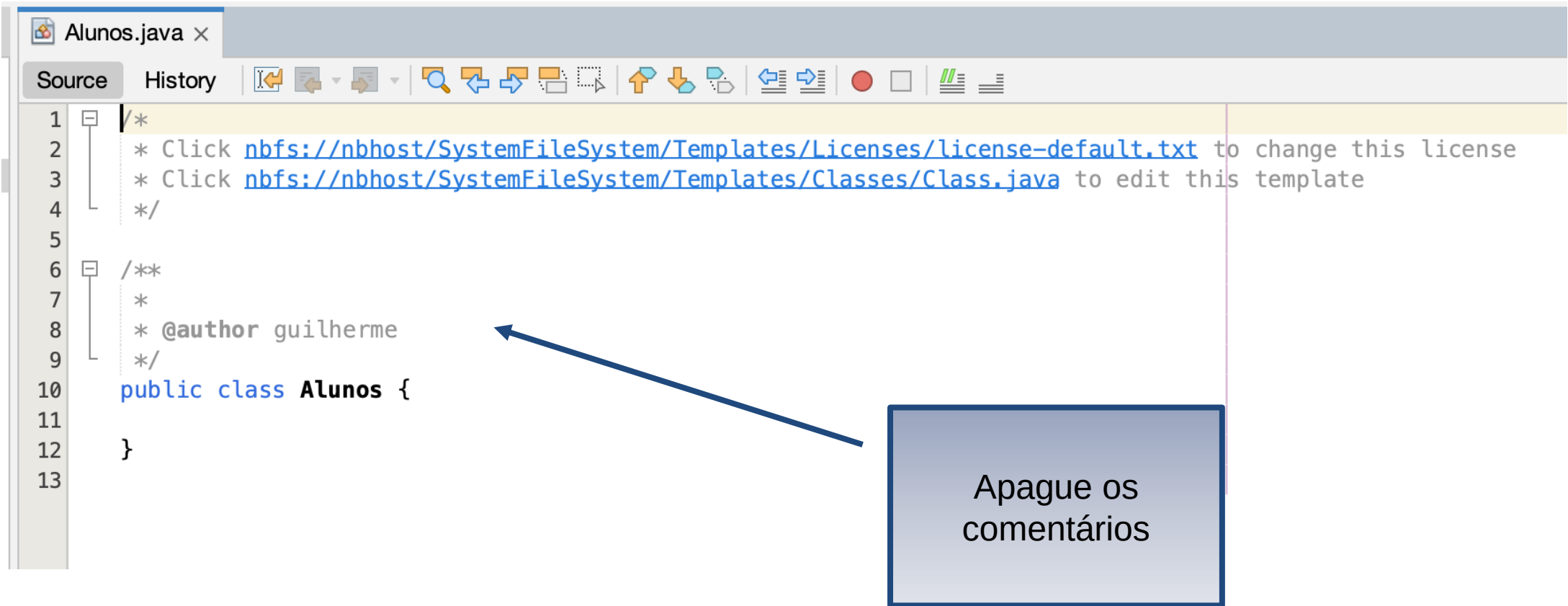
Warning: It is highly recommended that you do not place Java classes in the default package

< Back Next > **Finish** Cancel Help

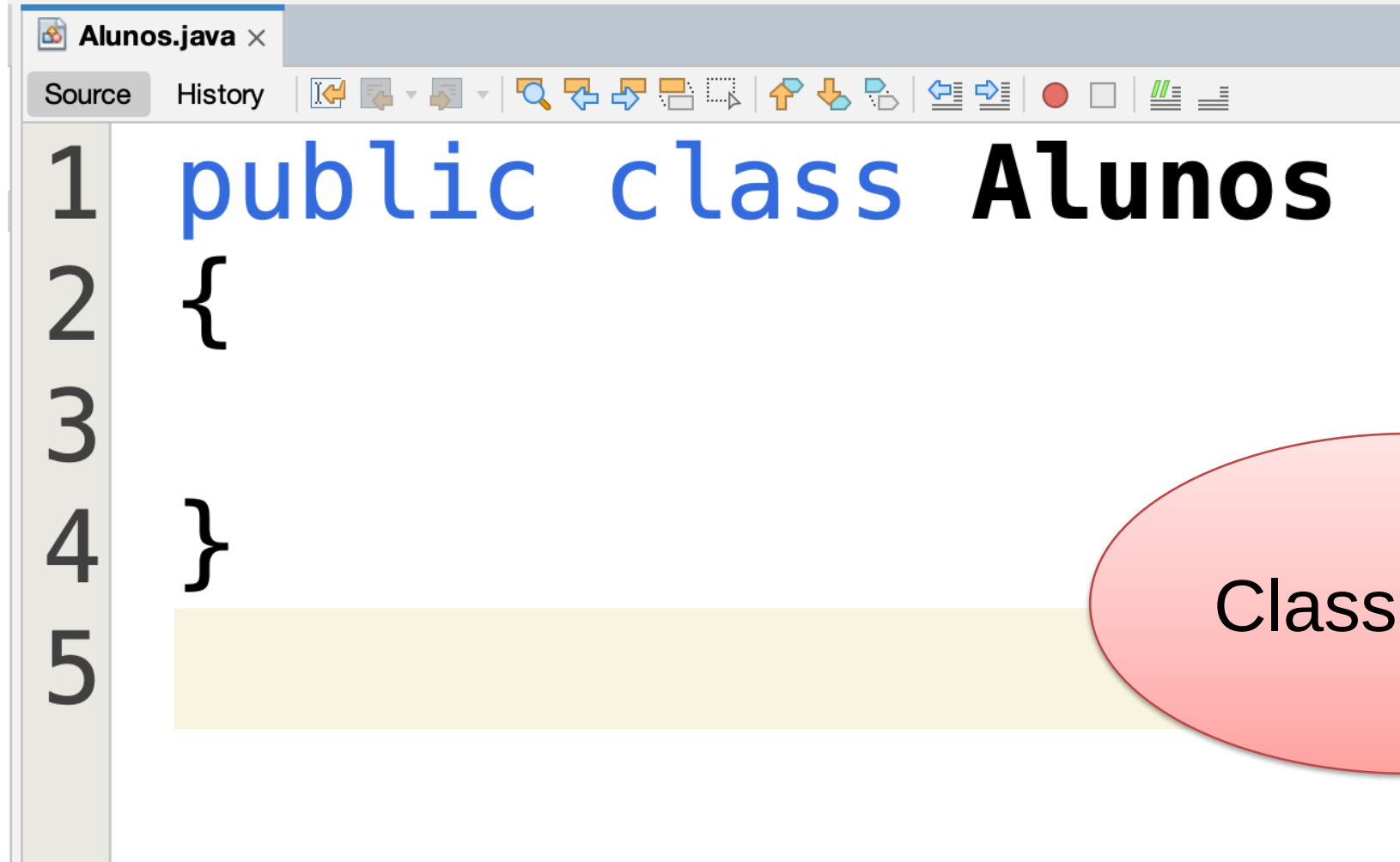
Digite o nome
Alunos

clique

Adicionando uma Classe ao Projeto



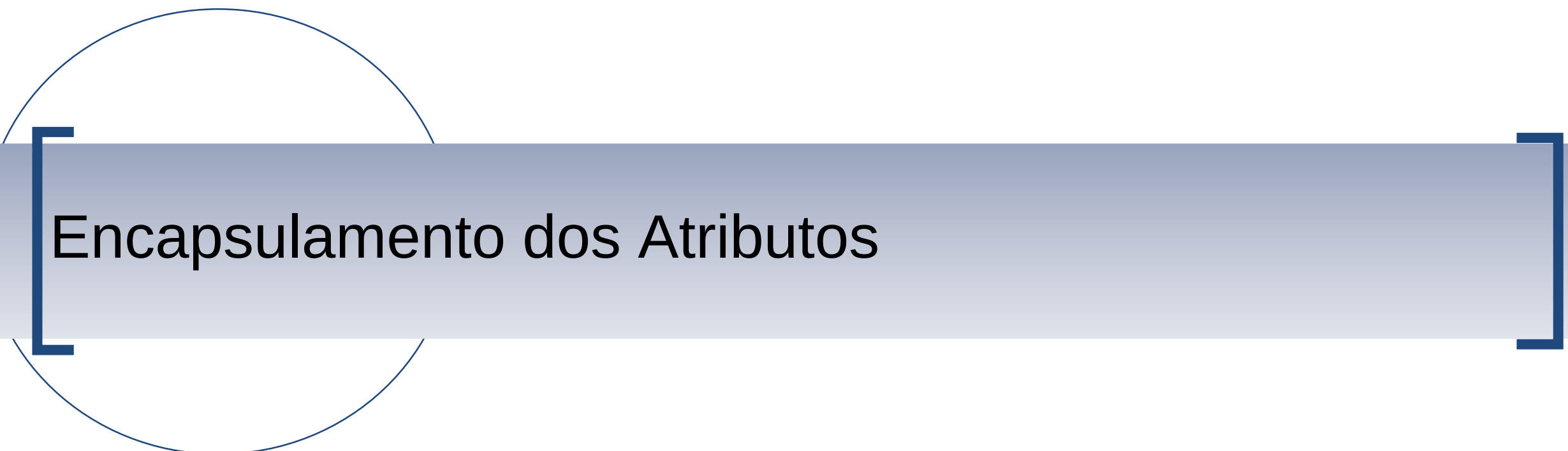
Adicionando uma Classe ao Projeto



The image shows a screenshot of an IDE window titled "Alunos.java". The window has a toolbar with various icons for editing and navigation. The code editor displays the following Java code:

```
1 public class Alunos
2 {
3
4 }
5
```

The code is highlighted in blue. A red oval callout points to the code, containing the text "Classe Java".

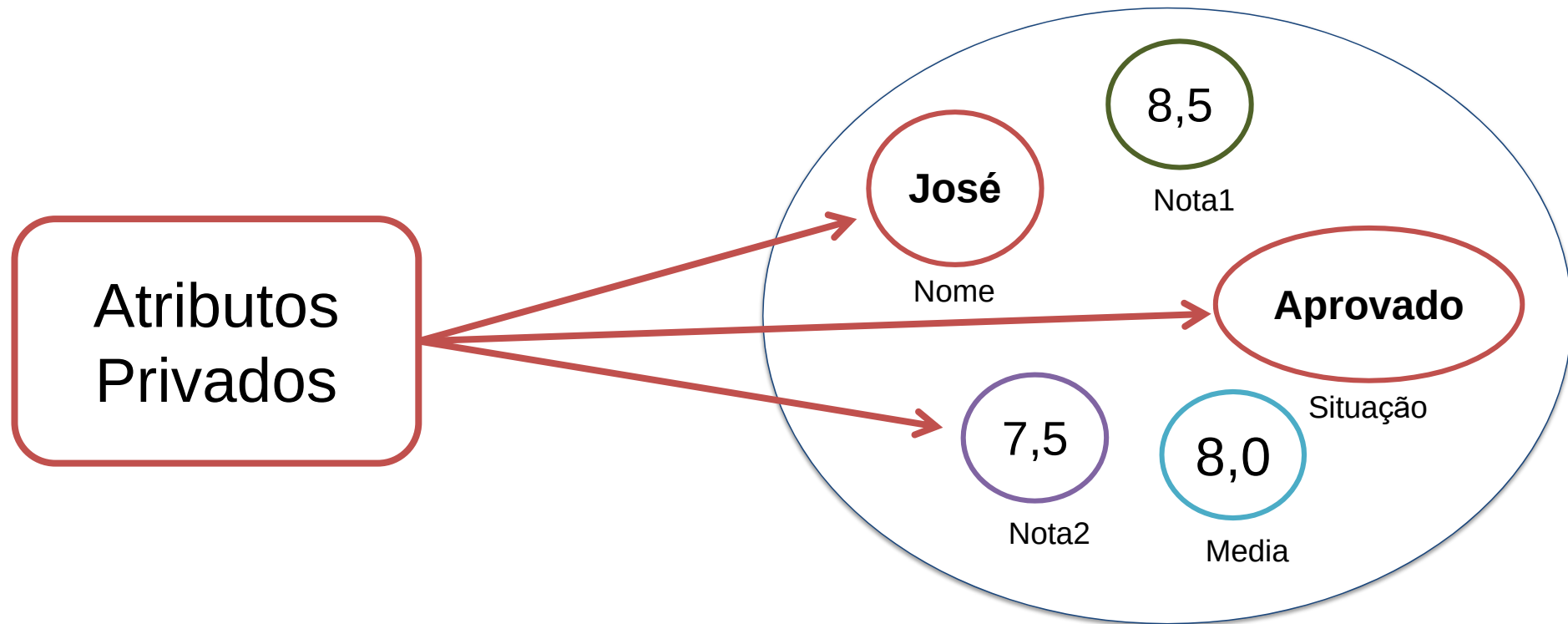
A decorative graphic consisting of a thin blue circle on the left and a thick blue horizontal bar extending across the top of the slide. The title text is centered within the bar.

Encapsulamento dos Atributos

Encapsulamento

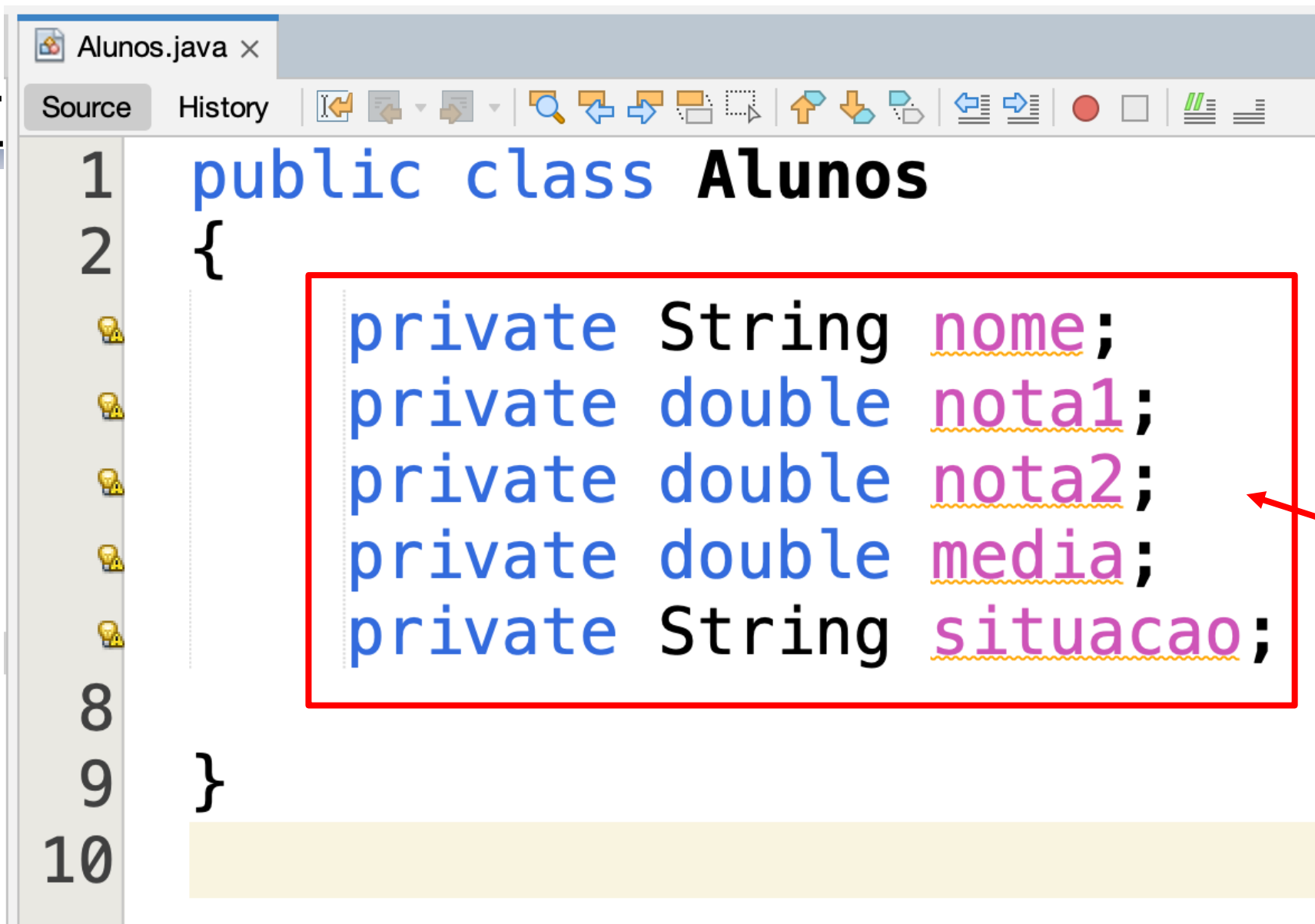
- O que é ?
 - O encapsulamento é um conceito importante na POO que envolve o empacotamento dos atributos e métodos relacionados em uma única unidade, a classe.
 - Isso limita o acesso direto aos atributos de um objeto e permite um maior controle sobre o acesso a eles.
 - Portanto será necessário estabelecer métodos de acesso para ler ou alterar um atributo

Encapsulamento



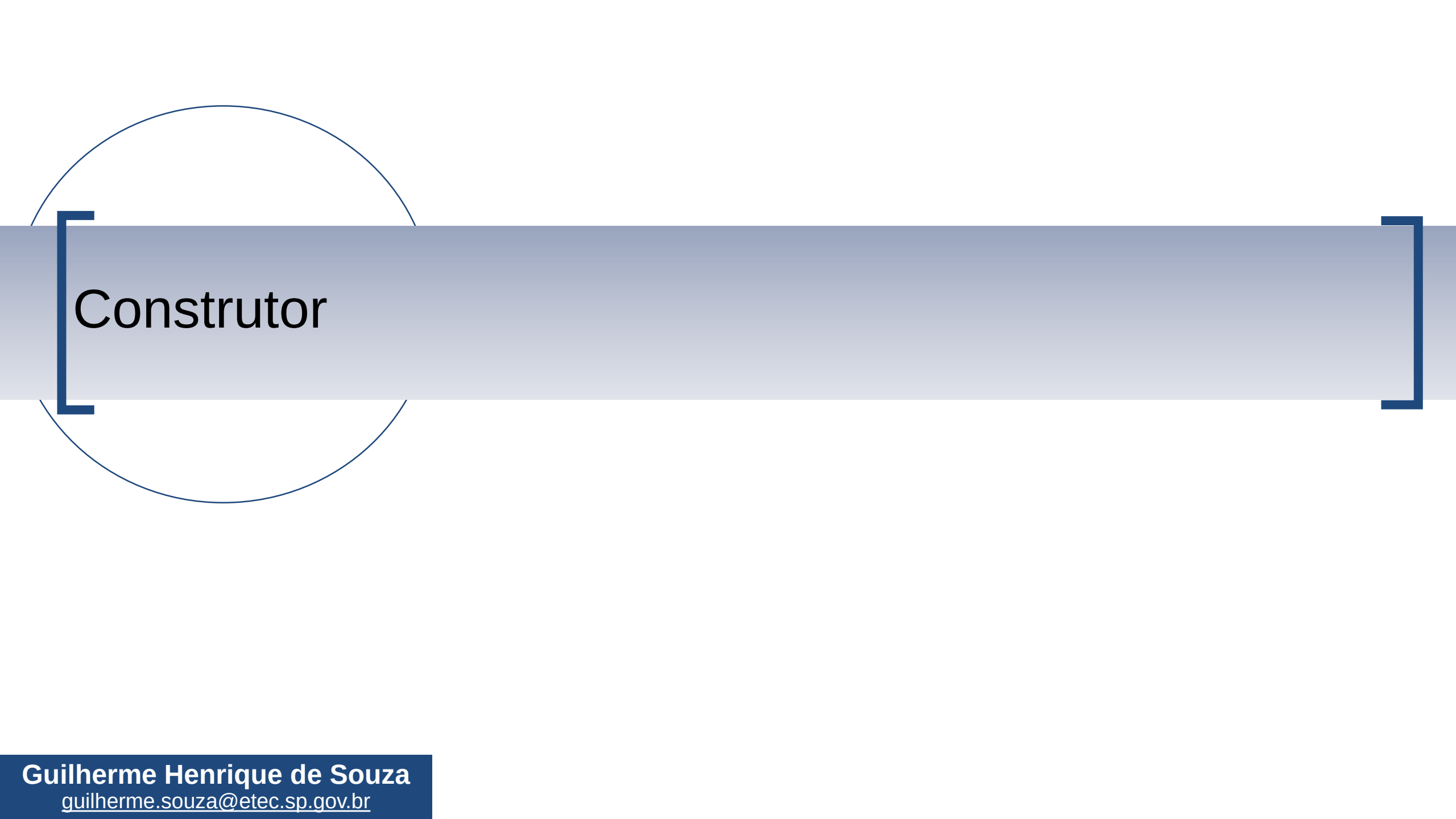
Encapsulando os atributos de uma classe

- A palavra-chave "*private*"
 - É utilizada para implementar o conceito de encapsulamento.
- O acesso direto a certos membros (atributos ou métodos) de uma classe será *restringido*.
- Quando um membro é declarado como "private", ele só pode ser acessado dentro da própria classe
 - Isso ajuda a manter a integridade dos dados e controlar seu acesso.



```
1 public class Alunos
2 {
3     private String nome;
4     private double nota1;
5     private double nota2;
6     private double media;
7     private String situacao;
8
9 }
10
```

Digite este
código Java

A decorative graphic consisting of a light blue horizontal bar spanning the width of the slide. On the left side, a thin blue circle is partially visible, with a thick blue bracket-like shape superimposed over it. The word "Construtor" is written in black text within the light blue bar.

Construtor

Construtor

- O que é um Construtor na POO ?
 - Também conhecido como "método construtor"
 - É um tipo especial de método em programação orientada a objetos que é chamado automaticamente quando um objeto de uma classe é criado.
 - Sua principal função é inicializar os atributos (dados) da classe e realizar quaisquer outras ações necessárias para garantir que o objeto seja criado em um estado válido e consistente.

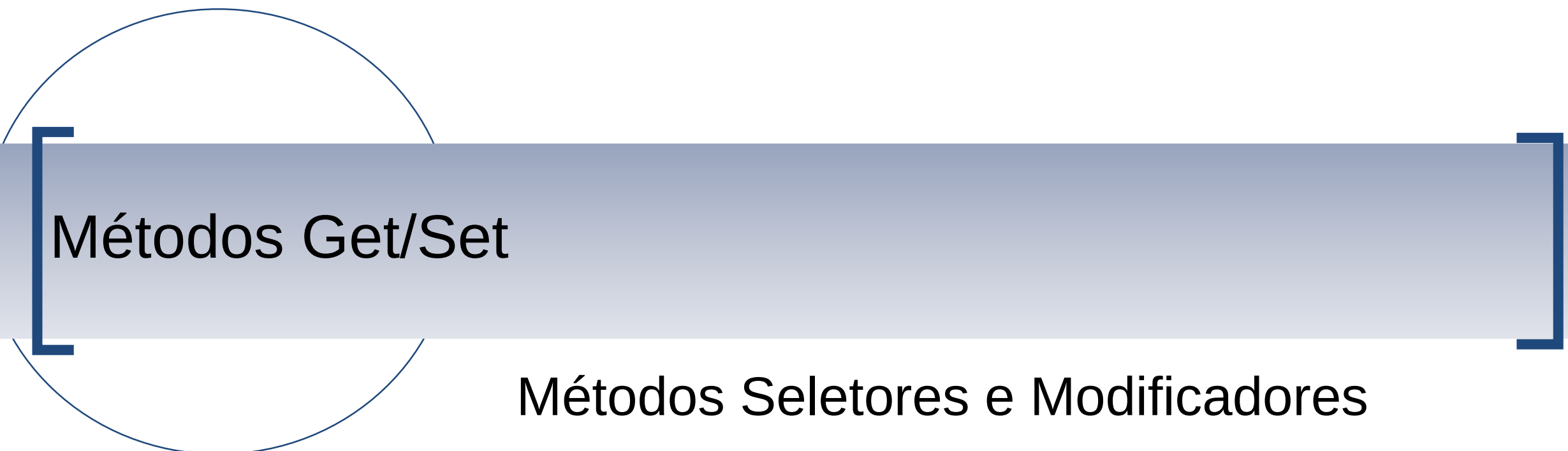
Construtor

- Principais Características de Construtores no Java
 - **Nome do método igual ao nome da classe:**
 - Um construtor tem o mesmo nome que a classe a que pertence. Isso é fundamental para que seja chamado automaticamente durante a criação do objeto.
 - **Não possui tipo de retorno:**
 - Diferentemente de outros métodos, o construtor não possui um tipo de retorno explícito.
 - **Chamado automaticamente:**
 - O construtor é chamado automaticamente no momento da criação de um objeto da classe, seja de forma explícita ou implícita.

Construtor

- Vamos declarar o método Construtor para a Classe Alunos.
- Note que ele tem o mesmo nome da classe, e não possui parâmetros de entrada.
- O método terá a função de inicializar alguns ou todos os atributos da classe.
- Entretanto: métodos construtores podem possuir um ou mais parâmetros de entrada. E uma classe pode possuir mais de um método construtor.

```
Alunos.java x
Source History
1 public class Alunos
2 {
3     private String nome;
4     private double nota1;
5     private double nota2;
6     private double media;
7     private String situacao;
8
9     public Alunos()
10    {
11        nome = "";
12        nota1 = 0;
13        nota2 = 0;
14        media = 0;
15        situacao = "nao informada";
16    }
17
18 }
19
```



Métodos Get/Set

Métodos Seletores e Modificadores

Métodos Get/Set

■ Métodos Seletores

- São métodos que **retornam** um valor.
- São usados para obter informações ou dados de um objeto.
- Por exemplo, um método seletor pode ser usado para retornar:
 - O nome de um objeto
 - O valor de um **atributo**
 - O resultado de uma operação

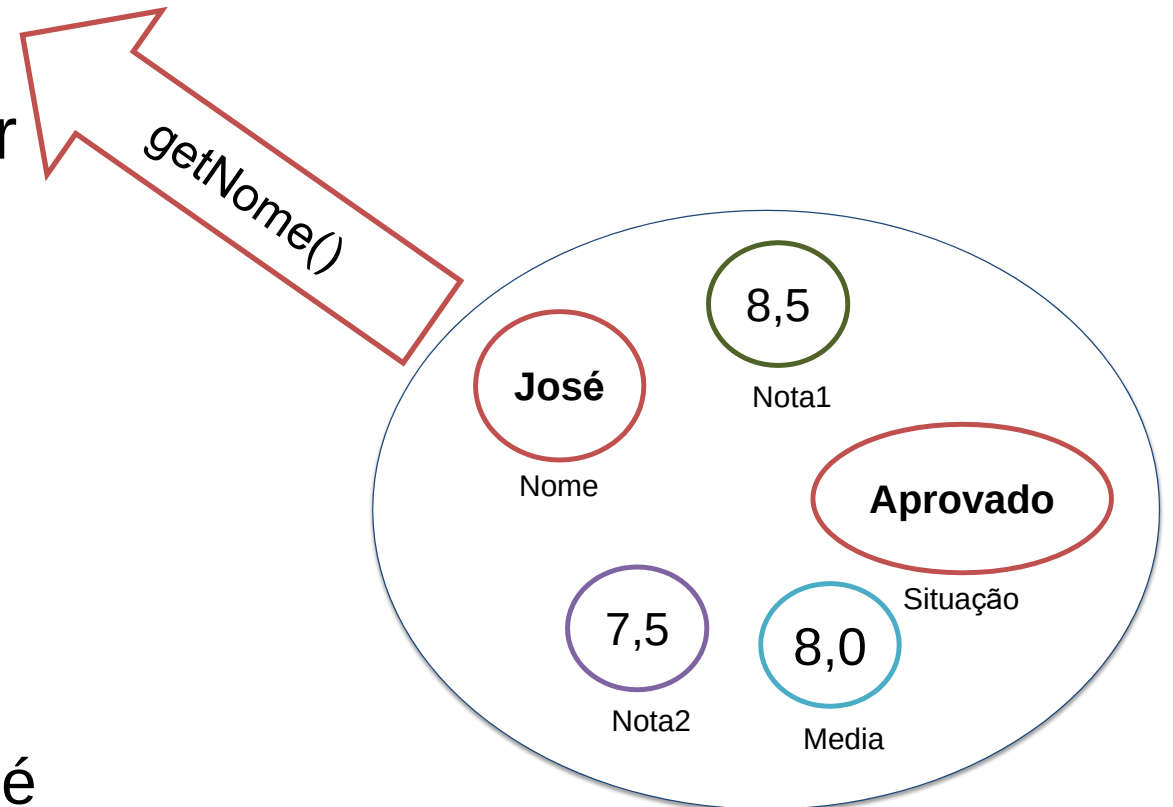
Métodos Get/Set

■ Métodos Seletores

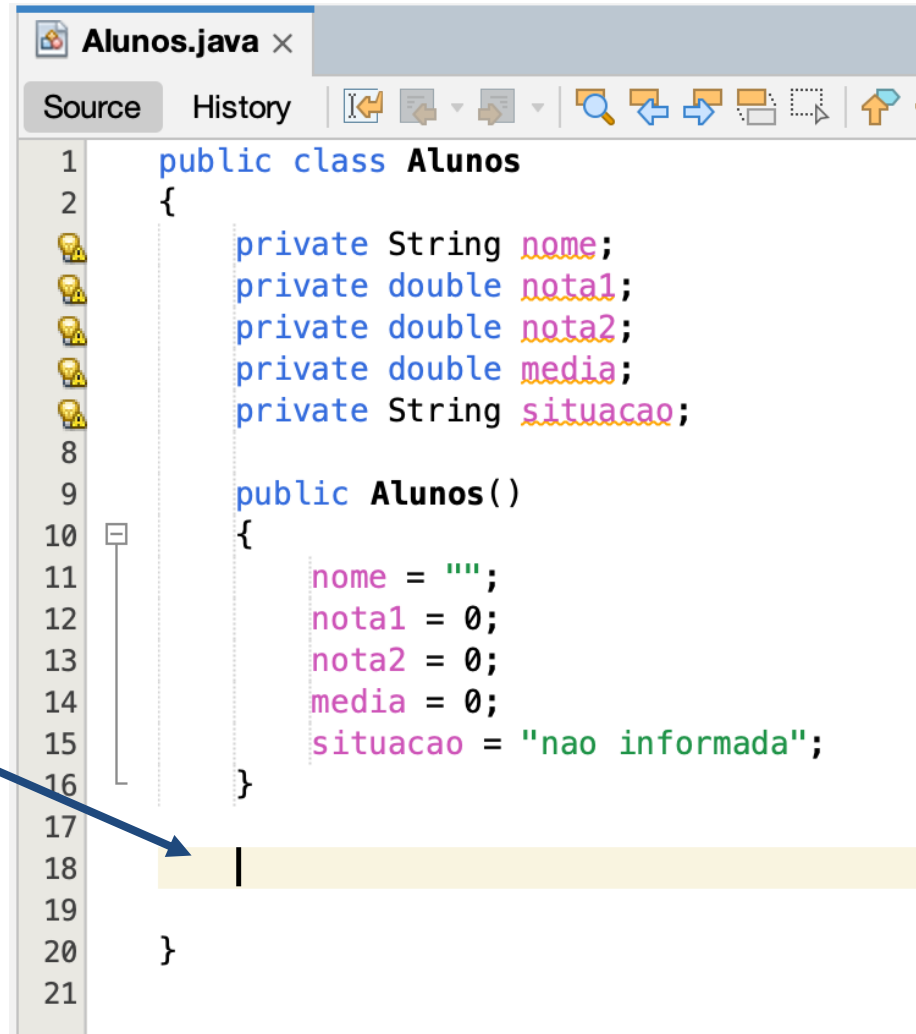
- Vamos declarar um método seletor para **retornar (get)** o valor do atributo nome

- O método `getNome()`

- Será de escopo **public** para que seja acessível externamente ao objeto.
- Terá como **retorno uma String** que é o mesmo tipo do atributo nome.



Métodos Get/Set



```
1 public class Alunos
2 {
3     private String nome;
4     private double nota1;
5     private double nota2;
6     private double media;
7     private String situacao;
8
9     public Alunos()
10    {
11        nome = "";
12        nota1 = 0;
13        nota2 = 0;
14        media = 0;
15        situacao = "nao informada";
16    }
17
18
19
20 }
21
```

Posicione o cursor do mouse
aqui neste ponto do seu
código

Métodos Get/Set

Declare o método getNome

Este método não possuirá parâmetros de entrada, terá como retorno uma String, e será do tipo public

```
Alunos.java x
Source History
1 public class Alunos
2 {
3     private String nome;
4     private double nota1;
5     private double nota2;
6     private double media;
7     private String situacao;
8
9     public Alunos()
10    {
11        nome = "";
12        nota1 = 0;
13        nota2 = 0;
14        media = 0;
15        situacao = "nao informada";
16    }
17
18    public String getNome()
19    {
20    }
21
22 }
23
24
```

Métodos Get/Set

O método deverá retornar o valor do atributo nome.

Veja que utilizamos o comando return para retornar um atributo (variável) da classe

```
    nome = "",  
    media = 0;  
    situacao = "nao informada";  
}  
  
public String getNome()  
{  
    return this.nome;  
}  
}
```

O ponteiro this faz referente a variáveis e métodos da própria classe.

Métodos Get/Set

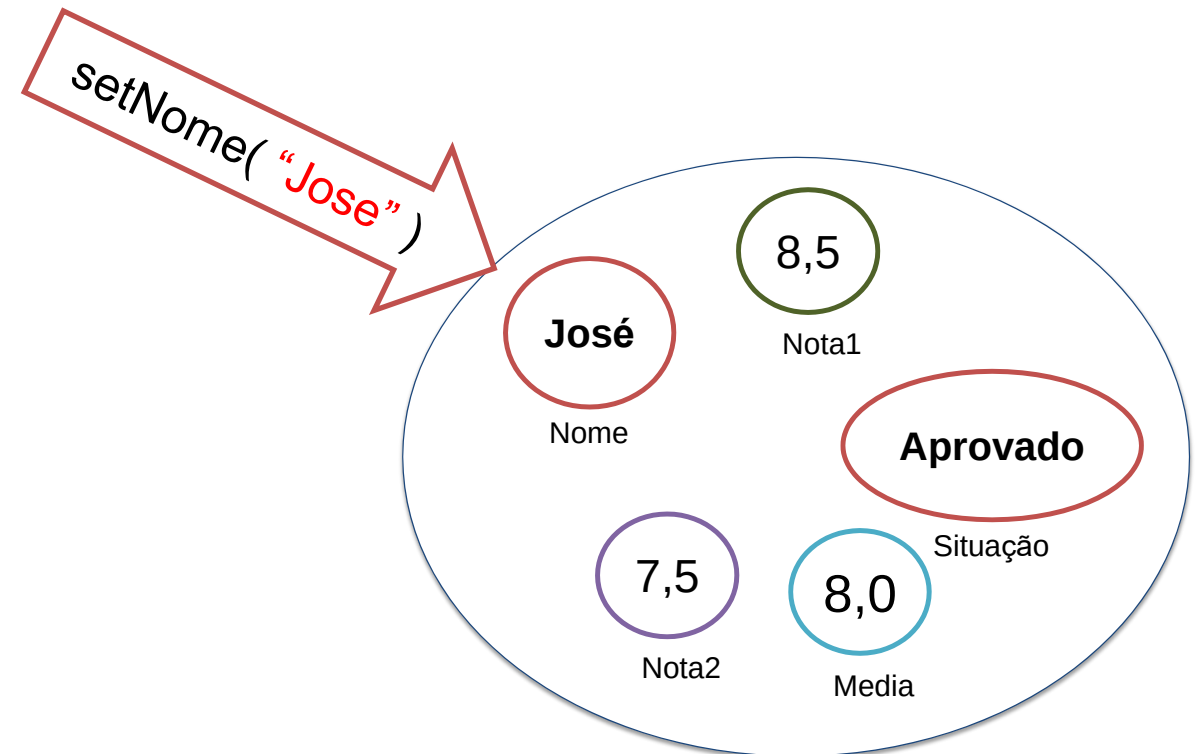
■ Métodos Modificadores

- São usados para alterar o estado de um objeto
- São usados para:
 - Atribuir novos valores a atributos
 - Realizar operações no estado de um objeto
 - Alterar o comportamento de um objeto.

Métodos Get/Set

■ Métodos Modificadores

- Vamos declarar um método modificador para **alterar (set)** o valor do atributo nome
- O método setName()
 - Será de escopo **public** para que seja acessível externamente ao objeto.
 - Não retornará valor, mas deverá ter como **retorno um void**.
 - **Terá um parâmetro de entrada que irá alterar o valor do atributo nome.**



Métodos Get/Set

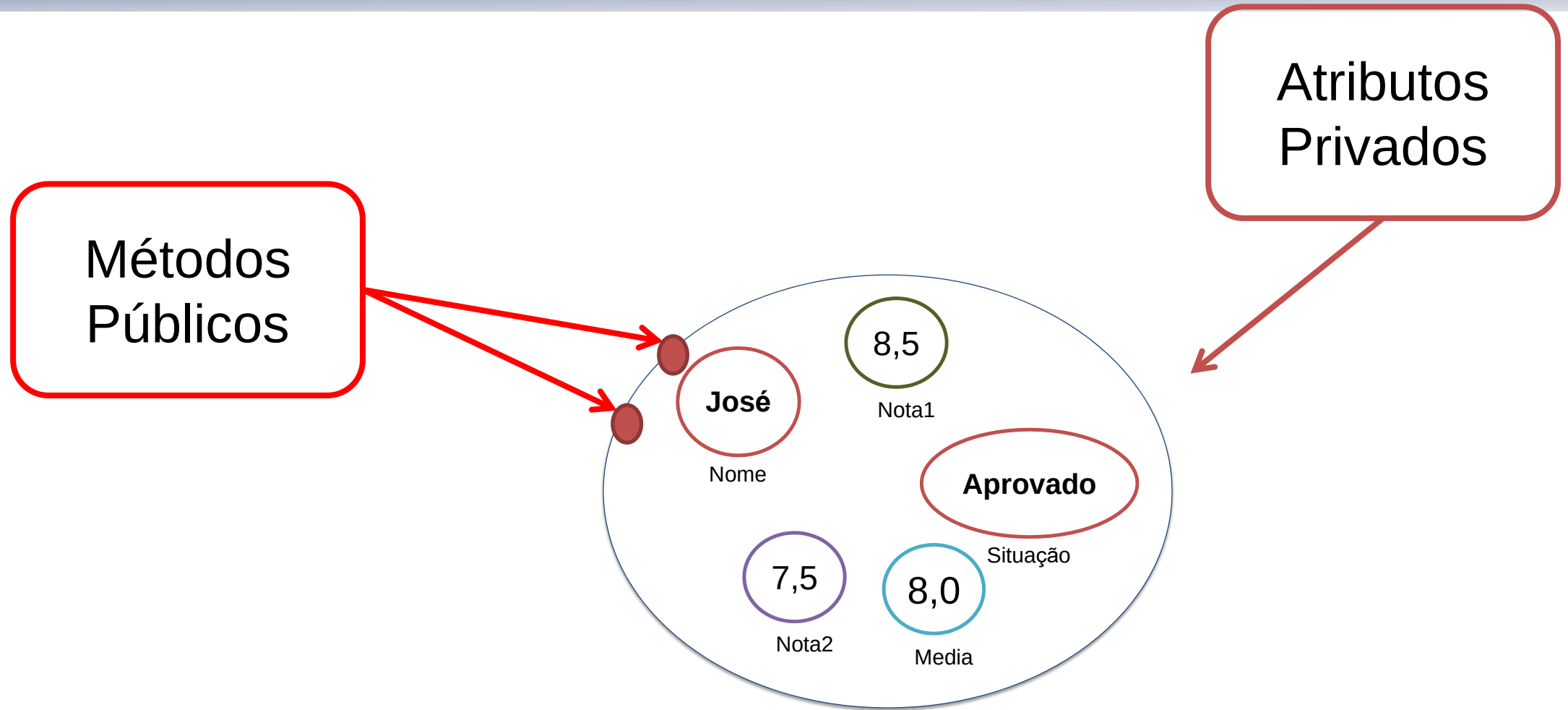
Declare o método setName

E atribua a variável nome ao atributo nome da classe. Utilize o ponteiro this para acessar o atributo.

```
    situacao = "nao informada";  
}  
  
public String getNome()  
{  
    return this.nome;  
}
```

```
public void setName(String nome)  
{  
    this.nome = nome;  
}
```

Métodos Get/Set



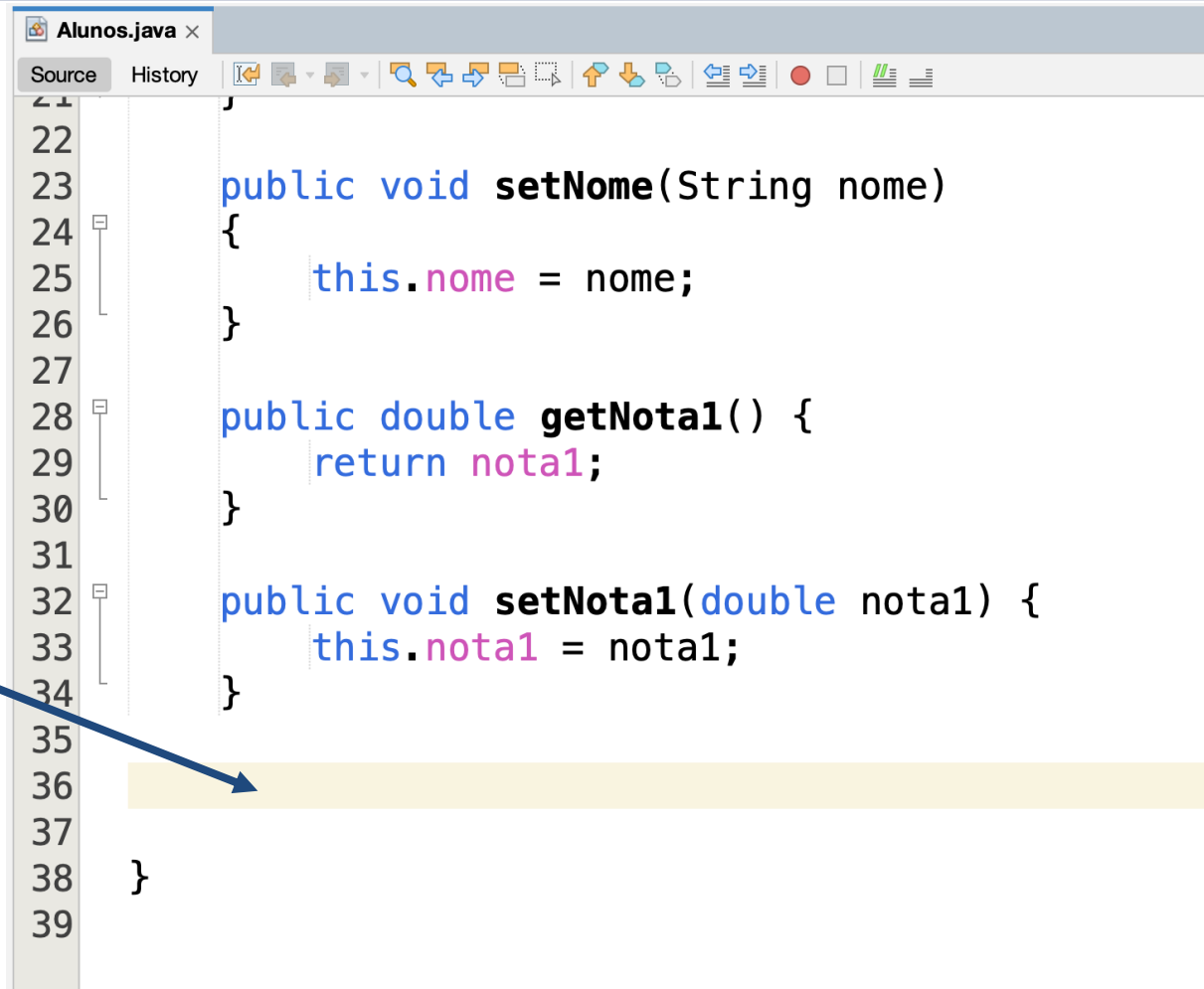
Métodos Get/Set

Implemente o par de métodos para o atributo `nota1`

```
}  
  
public double getNota1() {  
    return nota1;  
}  
  
public void setNota1(double nota1) {  
    this.nota1 = nota1;  
}  
  
}
```

Métodos Get/Set

Posicione o cursor
do mouse aqui

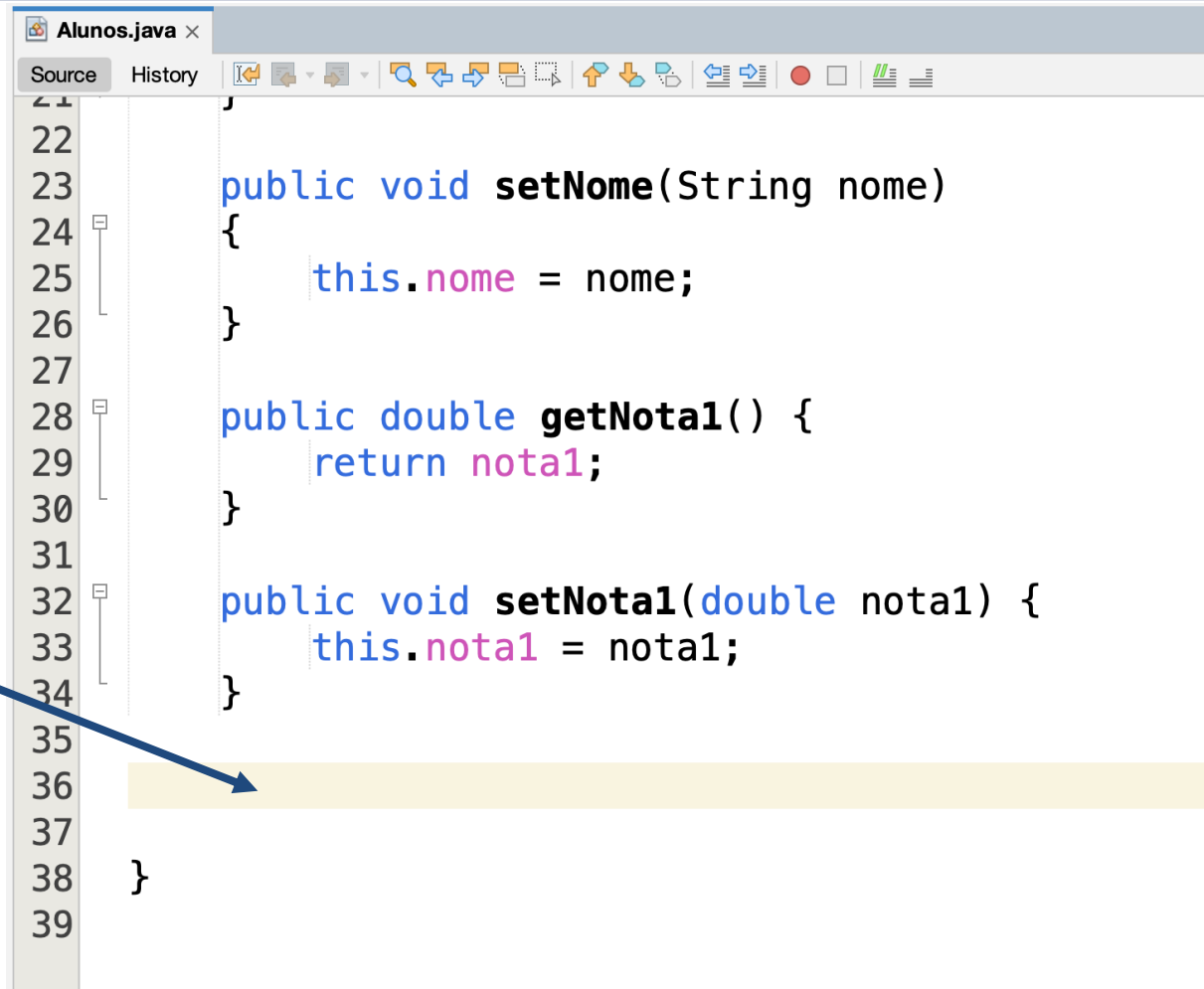


```
Alunos.java x
Source History
21
22
23 public void setNome(String nome)
24 {
25     this.nome = nome;
26 }
27
28 public double getNota1() {
29     return nota1;
30 }
31
32 public void setNota1(double nota1) {
33     this.nota1 = nota1;
34 }
35
36
37
38 }
39
```

Métodos Get/Set

Pressione no teclado

CTRL + Espaço



```
Alunos.java x
Source History
21
22
23 public void setNome(String nome)
24 {
25     this.nome = nome;
26 }
27
28 public double getNota1() {
29     return nota1;
30 }
31
32 public void setNota1(double nota1) {
33     this.nota1 = nota1;
34 }
35
36
37
38 }
39
```

```

31
32 public void setNota1(double nota1) {
33     this.nota1 = nota1;
34 }
35
36
37
38
39

```

Alunos(String nome, double nota1, double nota2, double media, String situacao) - generate

clone() - override Object

equals(Object obj) - override boolean

finalize() - override void

getMedia() - generate double

getNota2() - generate double

getSituacao() - generate String

hashCode() - override int

setMedia(double media) - generate void

setNota2(double nota2) - generate void

setSituacao(String situacao) - generate void

toString() - override String

@Test... junit

@Test... testng

public static void main(String[] args) {... main

public static void main(String[] args) {... psvm

abstract

Escolha o
getNota2()

Imported Items; Press 'Ctrl+SPACE' Again for All Items

Métodos Get/Set

```
L  
    }  
  
    public double getNota2() {  
        return nota2;  
    }  
  
}
```

Método getNota2
preenchido
automaticamente pela IDE
do Netbeans

Métodos Get/Set

Repita o procedimento para o setNota2

Repita o procedimento para o getMedia e getSituacao

Estes métodos não irão possuir set.

```
public void setNota1(double nota1) {
    this.nota1 = nota1;
}
```

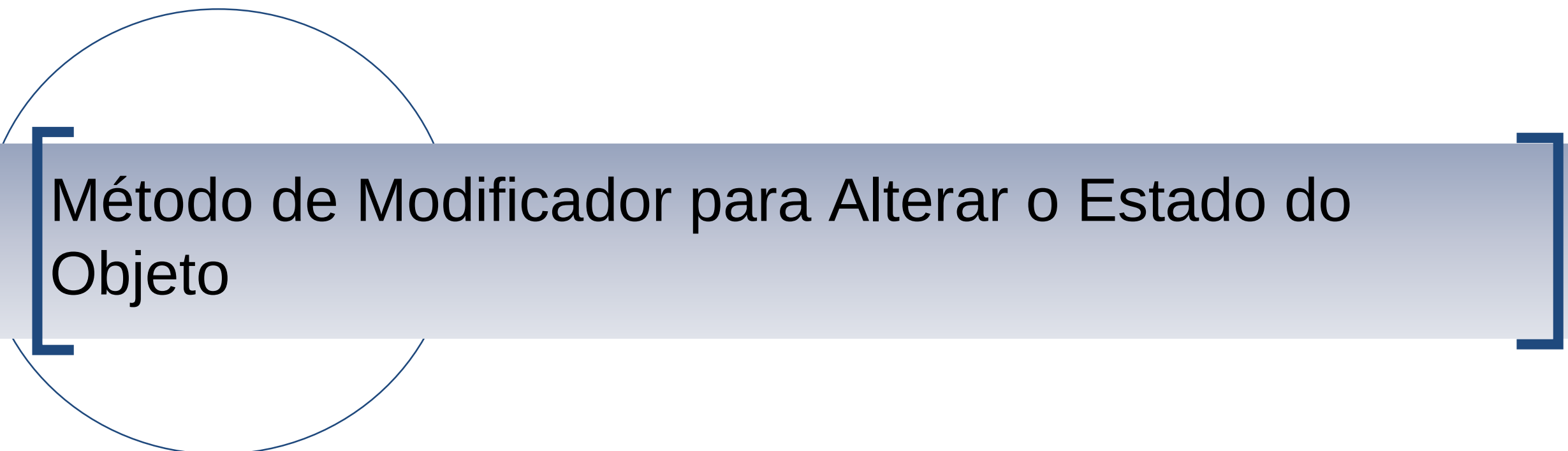
```
public double getNota2() {
    return nota2;
}
```

```
public void setNota2(double nota2) {
    this.nota2 = nota2;
}
```

```
public double getMedia() {
    return media;
}
```

```
public String getSituacao() {
    return situacao;
}
```

```
}
```

A decorative graphic consisting of a thin blue circle on the left side and a thick blue horizontal bar across the middle of the slide. The title text is centered within the blue bar.

Método de Modificador para Alterar o Estado do Objeto

Método de Modificador para Alterar o Estado do Objeto

- A classe Alunos
 - A classe que estamos implementando irá representar o objeto alunos
 - Na abstração utilizada no exemplo, todo aluno possui como característica (atributos)
 - Nome
 - Nota1
 - Nota2
 - Média
 - Situação



Método de Modificador para Alterar o Estado do Objeto

■ A classe Alunos

- O valor do atributo média será calculado a partir da média aritmética dos atributos **nota1** e **nota2**.
- O valor do atributo situação será obtido através da verificação se a média for superior ou igual a 6.0, e terá o valor **aprovado**, senão o valor será **reprovado**.



Método de Modificador para Alterar o Estado do Objeto

■ A classe Alunos

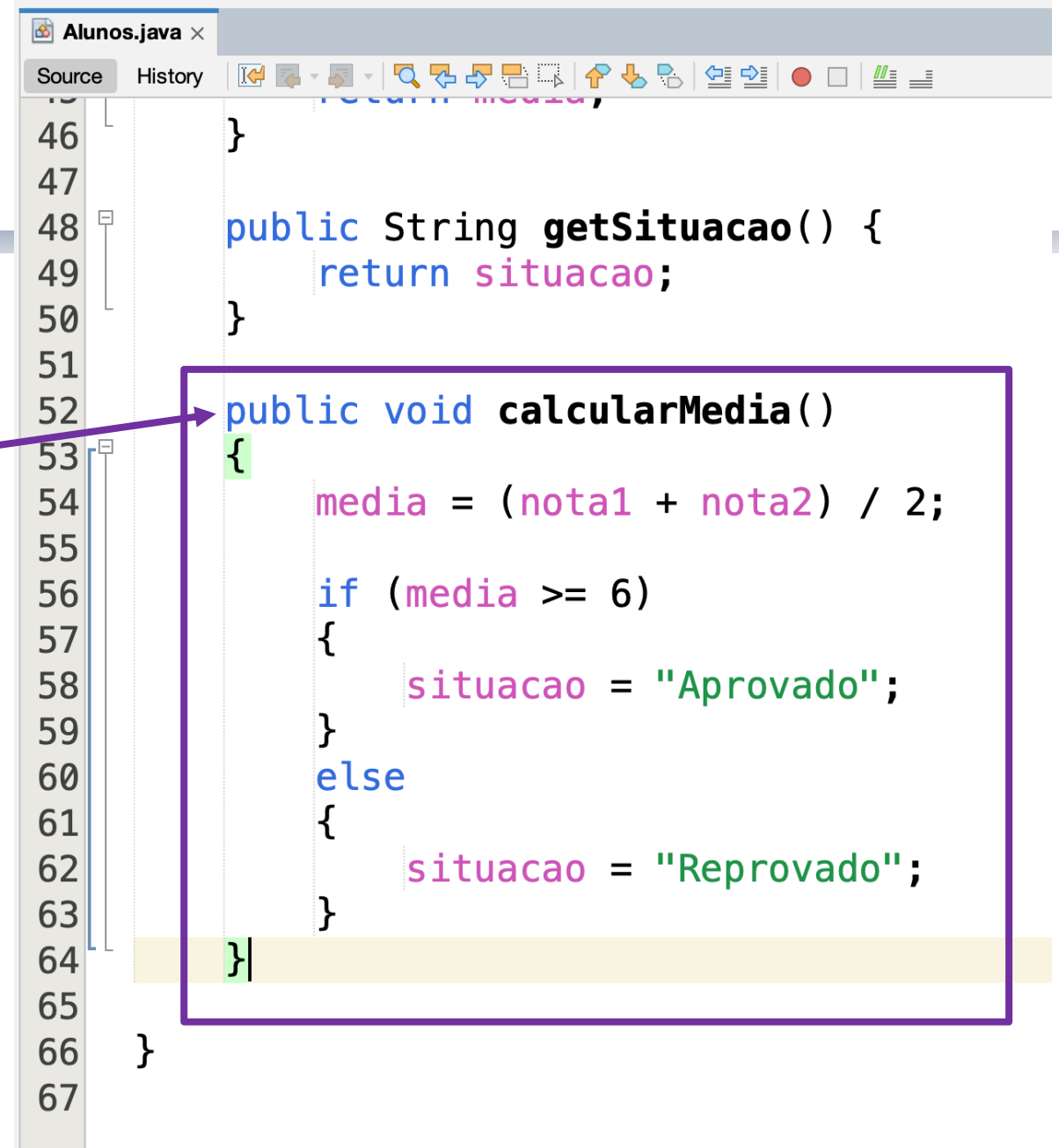
- Vamos implementar um método modificador que irá calcular o valor do atributo média e do atributo situação.
- Portanto, o objeto terá o seu estado alterado por um método modificador.
 - Este método não irá possuir parâmetro de entrada, pois irá manipular os atributos da classe, e também não irá retornar valor pois o resultado ficará nos atributos da classe.



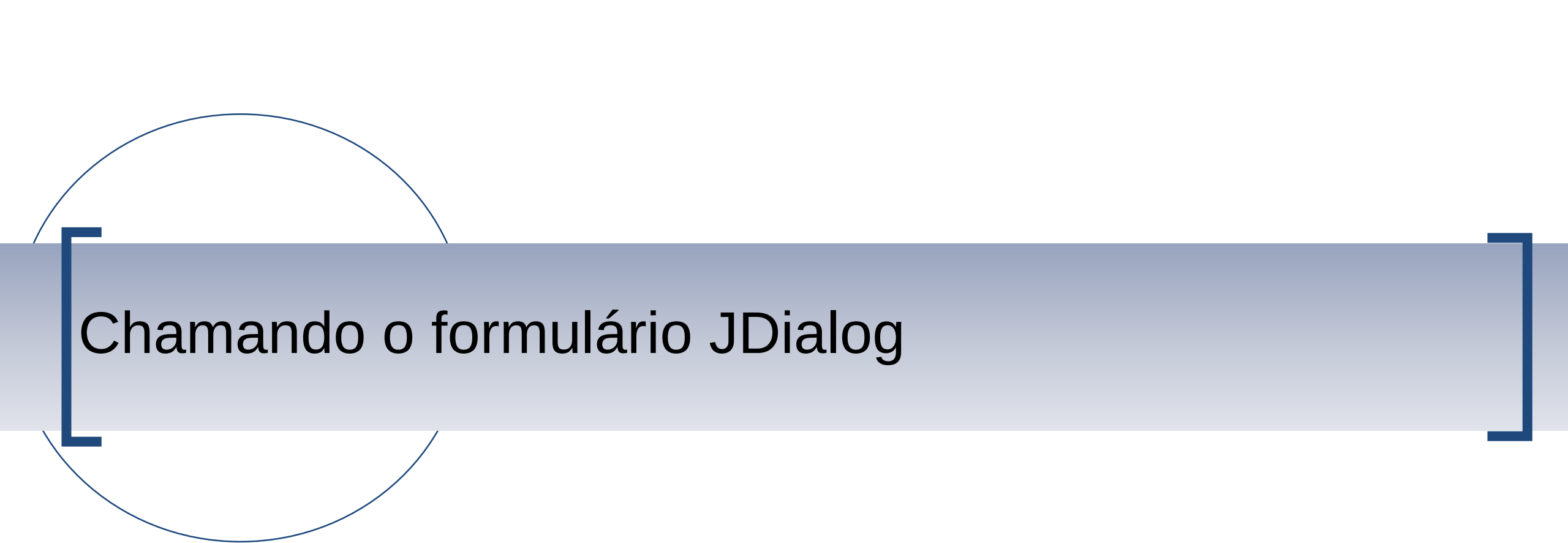
Método calcularMedia()

Implemente o método
calcularMedia()

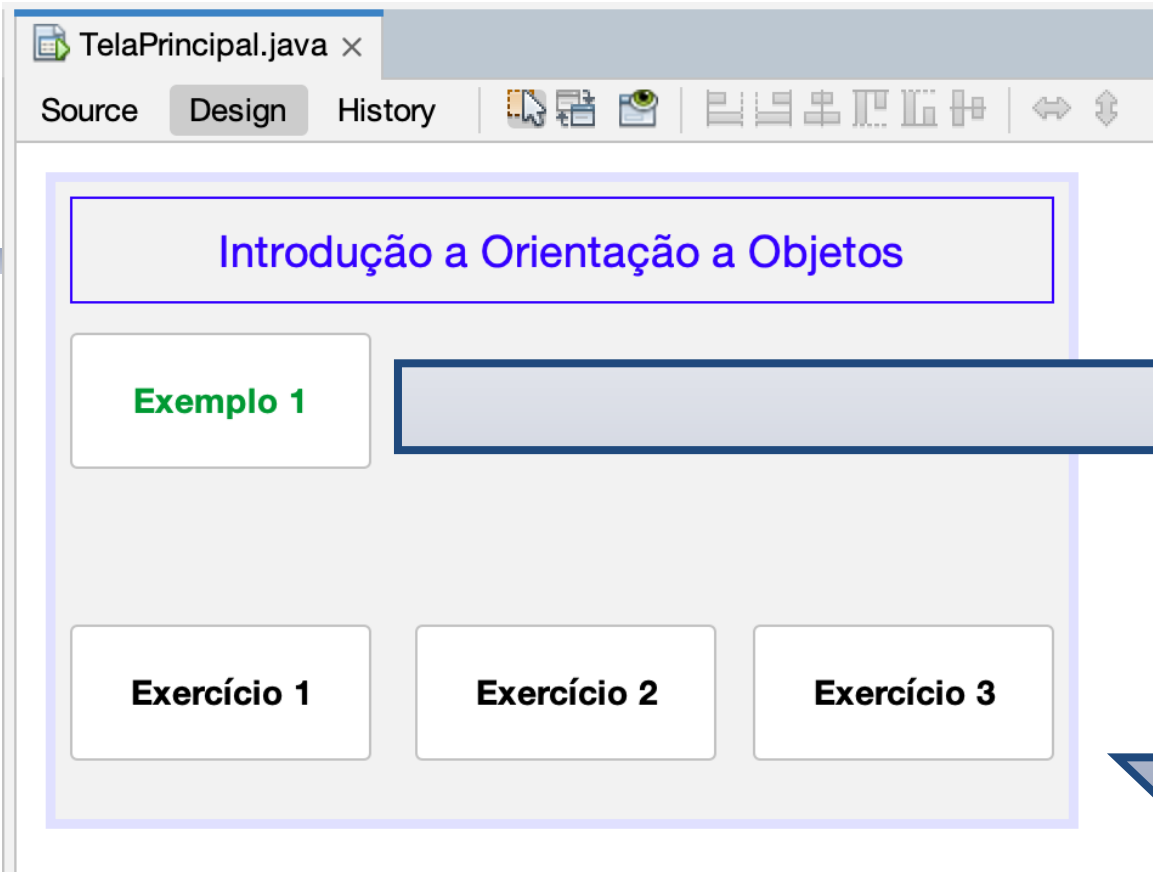
Este método não irá possuir parâmetro de entrada, pois irá manipular os atributos da classe, e também não irá retornar valor pois o resultado ficará nos atributos da classe.



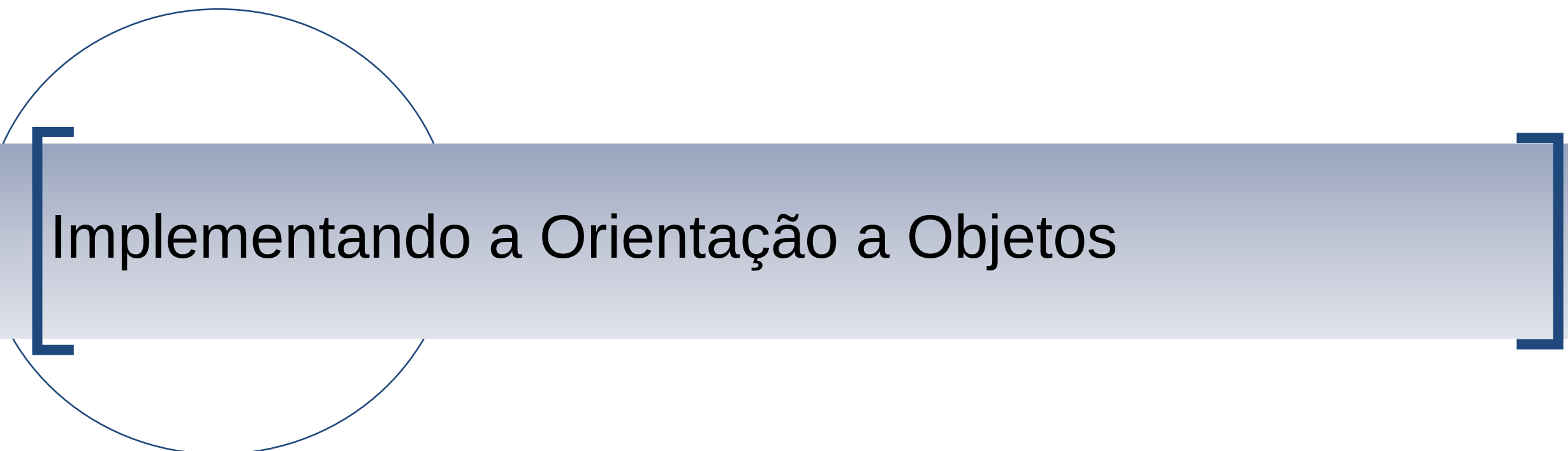
```
Alunos.java x
Source History
46 }
47
48 public String getSituacao() {
49     return situacao;
50 }
51
52 public void calcularMedia()
53 {
54     media = (nota1 + nota2) / 2;
55
56     if (media >= 6)
57     {
58         situacao = "Aprovado";
59     }
60     else
61     {
62         situacao = "Reprovado";
63     }
64 }
65
66 }
67
```



Chamando o formulário JDialog



```
private void btnExemplo1ActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
  
    Exemplo1 janela = new Exemplo1( parent:null, modal:false);  
    janela.setVisible( b:true);  
}
```



Implementando a Orientação a Objetos



Exemplo 1 - Calculo da Média de um Aluno

Digite o nome do aluno

Digite a nota 1

Digite a nota 2

Calcular utilizando Programação Estruturada

Calcular utilizando a Programação Orientada a Objetos

Média **resultado**

Situação **resultado**

Clique 2x neste botão

```
private void btnCalcularOOActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:
```

```
/**  
 * @param args the command line arguments  
 */
```

Implementando a Orientação a Objetos

```
private void btnCalcular00ActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
  
    Alunos objeto = new Alunos();  
  
}
```

Declare um objeto da
classe Alunos.

Instancie o objeto utilizando
o construtor Alunos().

Implementando a Orientação a Objetos

```
private void btnCalcular00ActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:
```

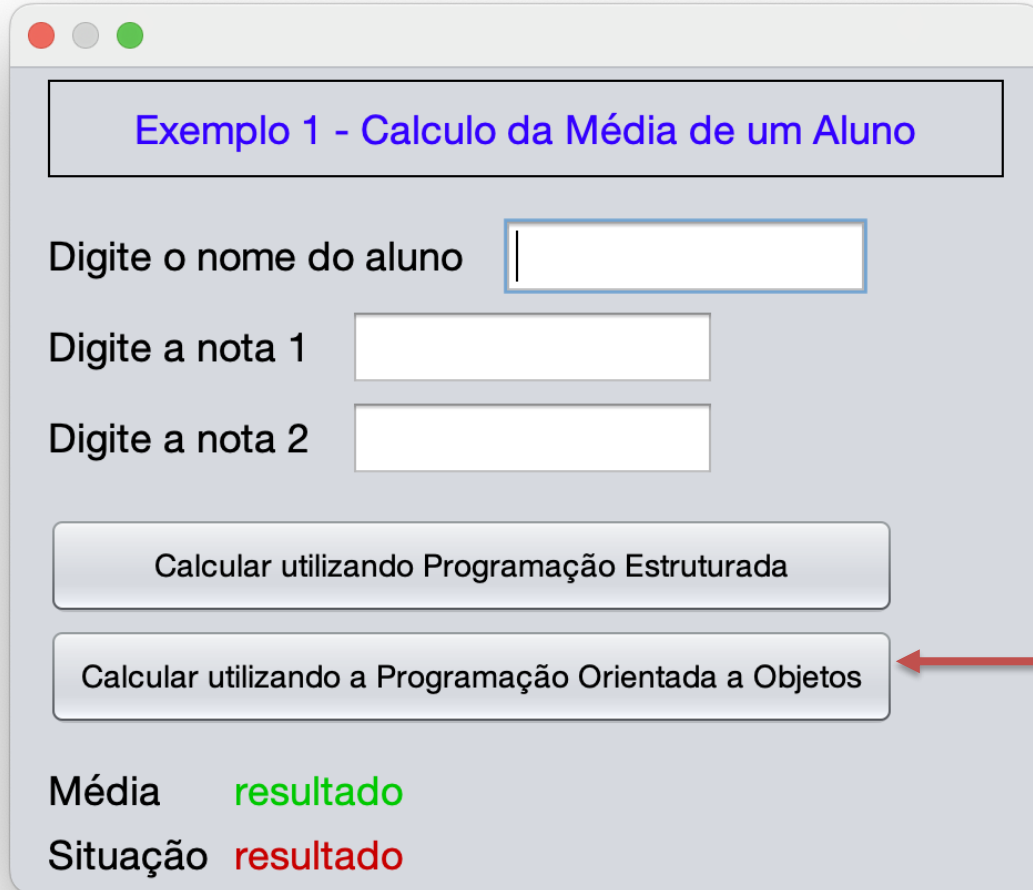
```
    Alunos objeto = new Alunos();
```

```
    labelSituacao.setText( texto:objeto.getSituacao() );  
    labelMedia.setText ( ""+ objeto.getMedia() );
```

```
}
```

Vamos implementar a chamada dos métodos seletores getSituacao e getMedia a partir do objeto

Implementando a Orientação a Objetos



Exemplo 1 - Calculo da Média de um Aluno

Digite o nome do aluno

Digite a nota 1

Digite a nota 2

Calcular utilizando Programação Estruturada

Calcular utilizando a Programação Orientada a Objetos

Média **resultado**

Situação **resultado**

Ao clicar no botão, será instanciado um novo objeto da Classe alunos.

Ao chamar o método construtor, serão atribuídos valores iniciais aos atributos da classe.

Implementando a Orientação a Objetos

Note que ao usarmos os métodos `getMedia` e `getSituacao()` retornamos o valor de ambos atributos para exibi-los nos respectivos labels

```
public Alunos()  
{  
    nome = "";  
    nota1 = 0;  
    nota2 = 0;  
    media = 0;  
    situacao = "nao informada";  
}
```

Exemplo 1 - Calculo da Média de um Aluno

Digite o nome do aluno

Digite a nota 1

Digite a nota 2

Calcular utilizando Programação Estruturada

Calcular utilizando a Programação Orientada a Objetos

Média 0.0

Situação nao informada

Implementando a Orientação a Objetos

```
private void btnCalcularActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
  
    Alunos objeto = new Alunos();  
  
    objeto.setNome( nome:txtNome.getText() );  
    objeto.setNota1( nota1:Double.parseDouble( s:txtNota1.getText() ) );  
    objeto.setNota2( nota2:Double.parseDouble( s:txtNota2.getText() ) );  
  
    objeto.calcularMedia();  
  
    labelSituacao.setText( text:objeto.getSituacao() );  
    labelMedia.setText ( ""+ objeto.getMedia() );  
}
```

Para os métodos setNome, setNota1, e setNota2 são passados como parâmetros os valores dos referidos campos de texto presente no formulários e fornecidos pelo usuário do programa.

Vamos implementar a chamada dos métodos modificadores setNome, setNota1, setNota2, e calcularMedia a partir do objeto.

Implementando a Orientação a Objetos

Exemplo 1 - Calculo da Média de um Aluno

Digite o nome do aluno

Digite a nota 1

Digite a nota 2

Calcular utilizando Programação Estruturada

Calcular utilizando a Programação Orientada a Objetos

Média **7.5**

Situação **Aprovado**

Exemplo 1 - Calculo da Média de um Aluno

Digite o nome do aluno

Digite a nota 1

Digite a nota 2

Calcular utilizando Programação Estruturada

Calcular utilizando a Programação Orientada a Objetos

Média **4.0**

Situação **Reprovado**



Exercícios

Exercício 1

- Criar uma **classe** no Java com o nome **Pessoas**
- Criar os **atributos**
 - **Nome** como texto
 - **Sexo** como texto
 - **Idade** como inteiro
 - **Meses** como inteiro
- Crie o **método construtor**, para inicializar os atributos
- Criar os seguintes **métodos get e set** para cada atributo
- Crie o método Calcular Idade em meses
 - Que deverá calcular a idade de anos para meses
- Crie um formulário JDialog com o nome **Exercicio1** que implemente a classe Pessoas

Introdução a Orientação a Objetos

Exemplo 1

Exercício 1

Exercício 2

Exercício 3

Exercício 1 - Aplicação da Classe Pessoa

Digite o nome

Informe o sexo

Digite a idade

Calcular utilizando a Programação Orientada a Objetos

A idade em meses é: **resultado**

Exercício 2

- Criar uma **classe** no Java com o nome **Medicos**
- Criar os **atributos**
 - **Nome** como texto
 - **Valor Consulta** como double
 - **Qtde Consulta** como integer
 - **Salario** como double
- Crie o **método construtor**, para inicializar os atributos
- Criar os seguintes **métodos get e set** para cada atributo
- Crie o método Calcular Salário
 - Que deverá calcular o salário multiplicando a quantidade pelo valor das consultas
 - e colocar o resultado no atributo salário
- Crie um formulário JDialog com o nome **Exercicio2** que implemente a classe **Medicos**

Exercício 3

- Criar uma **classe** no Java com o nome **Funcionarios**
- Criar os **atributos**
 - **Nome** como texto
 - **Salario Bruto** como double
 - **Valor do INSS** como double
 - **Salario Liquido** como double
- Crie o **método construtor**, para inicializar os atributos
- Criar os seguintes **métodos get e set** para cada atributo
- Crie o método Calcular Salário
 - Que deverá calcular o salário líquido subtraindo o valor do INSS considerando 8% do salário bruto
- Crie um formulário JDialog com o nome **Exercicio1** que implemente a classe **Funcionarios**